Self-Adaptive Evolutionary InfoVAE Framework for Hyperspectral Unmixing with Endmember Variability

A
Project Report

Submitted in partial fulfilment of the
Requirements for the award of the Degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

By

M Lokesh 1602-21-737-030

B Shiva Shankar 1602-21-737-050

Under the guidance of

Dr. Hitendra Sarma

Associate Professor

Department of Information Technology

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Ibrahimbagh, Hyderabad-31

2025

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology

DECLARATION BY THE CANDIDATE

We, **M Lokesh** and **B Shiva Shankar** bearing hall ticket numbers, **1602-21-737-030** and **1602-21-737-050** hereby declare that the project report entitled **Self-Adaptive Evolutionary InfoVAE Framework for Hyperspectral Unmixing with Endmember Variability** under the guidance of **Dr. Hitendra Sarma, Associate Professor**, Department of Information Technology, Vasavi College of Engineering, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Information Technology**

This is a record of bonafide work carried out by us and the results embodied in this project report have not been submitted to any other university or institute for the award of any other degree or diploma.

M. Lokesh
1602-21-737-030
B Shiva Shankar
1602-21-737-050

Vasavi College of Engineering (Autonomous)

ACCREDITED BY NAAC WITH 'A++' GRADE

(Affiliated to Osmania University)

Hyderabad-500 031

Department of Information Technology

BONAFIDE CERTIFICATE

This is to certify that the project entitled **Self-Adaptive Evolutionary InfoVAE Framework for Hyperspectral Unmixing with Endmember Variability** being submitted by **M Lokesh** and **B Shiva Shankar** bearing **1602-21-737-030** and **1602-21-737-050** in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by them under my guidance.

Dr. T. Hitendra Sarma
Associate Professor
Internal Guide

Dr. K. Ram Mohan Rao
Professor & HOD, IT
External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies that the successful completion of the Main project would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them.

It is with immense pleasure that we would like to take the opportunity to express our humble gratitude to **Dr. T. Hitendra Sarma, Associate Professor**, **Department of Information Technology** under whom we executed this project. We are also grateful to **C. Sireesha, Assistant Professor, Department of Information Technology** for her guidance. Their constant guidance and willingness to share their vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks.

We are very much thankful to **Dr. K. Ram Mohan Rao, Professor & HOD**, **Department of Information Technology**, for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. S. V. Ramana, Principal** of **Vasavi College of Engineering and Management,** for providing facilities. Not to forget, we thank all other faculty and non-teaching staff, who had directly or indirectly helped and supported me in completing my project in time.

Abstract

Hyperspectral unmixing (HU) plays a crucial role in remote sensing and environmental monitoring by extracting endmembers and their abundances from mixed pixels. Traditional linear and some nonlinear models fall short in modeling the complex, real-world spectral variability. Even advanced machine learning models like VAEs struggle with capturing dynamic endmember variability influenced by environmental conditions. Existing research largely neglects integrating evolutionary algorithms with VAEs to enhance adaptability and performance across varying data characteristics. Moreover, maximizing mutual information in latent space and leveraging spatial-spectral correlations remain underexplored. To address these gaps, this study proposes a self-adaptive evolutionary framework, SA-eInfoVAE, that merges InfoVAE principles with evolutionary strategies for automated parameter optimization. This method dynamically adjusts to diverse datasets, enhancing both reconstruction accuracy and generalization. The proposed framework improves hyperspectral unmixing robustness and flexibility, enabling more precise endmember and abundance estimations in complex scenarios. It also supports efficient real-time analysis in fields like precision agriculture, environmental monitoring, and resource management, offering a versatile tool for the remote sensing and machine learning communities.

Table of Contents

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

S. No	Abbreviation	Full Form
1	VAE	Variational Auto Encoder
2	SU	Spectral Unmixing
3	SBX	Simulated Binary Crossover
4	SA eInfoVAE	Self-adaptive evolutionary Informational Variational Autoencoder
5	ANC	Abundance Non-Negativity Constraint
6	ASC	Abundance Sum Constraint
7	PGMSU	Probabilistic Generative Model for Spectral Unmixing
8	ReLU	Rectified Linear Unit
9	PCA	Principal Component Analysis
10	KL Divergence	Kullback-Leilbler (KL) divergence
11	MMD	Maximum Mean Discrepancy
12	SAD	Spectral Angle Divergence
13	RE	Reconstruction Error
14	RMSE	Root Mean Squared Error
15	RGB	Red, Green, Blue
16	SSD	Solid State Drive
17	GPU	Graphical Processing Unit

1. INTRODUCTION

1.1 Problem Statement – Overview

Hyperspectral unmixing (HU) is a fundamental problem in remote sensing, where the goal is to decompose mixed pixel spectra into their constituent endmembers and estimate their corresponding abundances. However, traditional linear models and even advanced nonlinear approaches, such as variational autoencoders (VAEs), face significant challenges when dealing with real-world hyperspectral data. One of the primary issues is the variability of endmembers caused by environmental factors such as atmospheric conditions, illumination changes, and surface heterogeneity. This variability often leads to poor generalization and inaccurate unmixing results. Furthermore, existing models do not sufficiently incorporate spatial and spectral correlations, which are essential for improving the unmixing accuracy in complex datasets. Additionally, many current methods overlook the potential of self-adaptive frameworks that can automatically adjust to the changing nature of the hyperspectral data. There is a critical gap in integrating evolutionary algorithms with VAEs to create adaptive models that can optimize their parameters dynamically in response to endmember variability and other environmental factors. The lack of such adaptive methods limits the accuracy and robustness of hyperspectral unmixing systems, especially in real-time applications. Therefore, the problem is to develop a self-adaptive evolutionary VAE framework that can effectively handle endmember variability, maximize mutual information in latent spaces, and leverage spatial and spectral correlations for more accurate and robust hyperspectral unmixing.

1.2 Motivation

Hyperspectral unmixing (HU) plays a vital role in remote sensing, particularly for applications such as environmental monitoring, land cover classification, and resource management. Despite significant advancements in machine learning techniques, existing methods for hyperspectral unmixing still face challenges due to the high variability of endmembers and the complex, non-linear nature of hyperspectral data. Traditional unmixing approaches rely on linear models that fail to capture the intricate relationships within hyperspectral data, leading to

suboptimal performance, especially in cases of endmember variability and environmental changes. Although advanced techniques such as variational autoencoders (VAEs) have shown promise, they often struggle with generalization and are limited in terms of adaptability to varying datasets.

The motivation for this work arises from the need for a more robust, flexible, and self-adaptive framework capable of addressing these issues. Current research predominantly overlooks the integration of evolutionary algorithms (EAs) with VAEs, which can dynamically optimize the model parameters to adapt to diverse hyperspectral data characteristics. The lack of methods that fully utilize the spatial and spectral correlations inherent in hyperspectral data further contributes to the gap in achieving accurate unmixing results. By combining self-adaptive evolutionary strategies with VAEs, this research seeks to fill these gaps, offering a solution that can optimize both reconstruction and generalization capabilities for improved unmixing accuracy. This work introduces the concept of the self-adaptive evolutionary InfoVAE (SA-eInfoVAE), which is designed to adjust model parameters in real-time, enhance the robustness of endmember extraction, and improve the overall unmixing performance in challenging environments. The integration of these techniques is expected to provide substantial benefits in hyperspectral image analysis, leading to more accurate and efficient systems for real-world applications in remote sensing and beyond.

1.3 Scope & Objectives of the Proposed Work

The scope of this proposed work is to develop a self-adaptive evolutionary framework for hyperspectral unmixing that integrates the InfoVAE model with evolutionary algorithms. This framework will be specifically designed to address the challenges associated with endmember variability, spatial and spectral correlations, and model adaptability in hyperspectral image analysis. The key focus will be on improving the accuracy, robustness, and generalization of hyperspectral unmixing techniques for diverse real-world datasets. The proposed work aims to enhance the performance of hyperspectral unmixing systems, particularly in complex environments where traditional methods struggle. The scope of this research also includes the integration of evolutionary optimization techniques to dynamically adjust model parameters for more efficient and accurate results.

The objectives of the proposed work are as follows:

1. **Development of the SA-eInfoVAE Framework**: To design and implement a self-adaptive evolutionary VAE model that optimizes both the reconstruction and generalization capabilities of hyperspectral unmixing systems.
2. **Handling Endmember Variability**: To address the issue of endmember variability in hyperspectral data, ensuring that the framework can adapt to changing environmental conditions and diverse datasets.
3. **Maximization of Mutual Information in Latent Space**: To incorporate InfoVAE techniques for maximizing mutual information in the latent space, enhancing the separability of the extracted features.
4. **Evaluation on Real-World Datasets**: To test the proposed framework on several real-world hyperspectral datasets, comparing its performance against existing methods to demonstrate its superior adaptability and unmixing accuracy.
5. **Implementation of Evolutionary Optimization**: To integrate evolutionary algorithms for the optimization of model parameters in real-time, ensuring that the system can adapt to diverse hyperspectral data characteristics and perform efficiently across varying environment

1.4 Organization of the Report

Chapter 1 introduces the problem statement, emphasizing the challenges of hyperspectral unmixing, particularly endmember variability and its impact on remote sensing applications. It also outlines the motivation, scope, and objectives of the proposed framework

Chapter 2 provides a detailed review of existing methodologies and technologies related to hyperspectral unmixing. It examines approaches such as traditional linear models, variational autoencoders (VAEs), and evolutionary algorithms, highlighting their strengths, limitations, and gaps in addressing real-world hyperspectral data challenges.

Chapter 3 introduces the SA-eInfoVAE framework, detailing its architecture, components, and workflow. The innovative combination of Informational VAEs with evolutionary algorithms is explained, along with its self-adaptive mechanisms to dynamically handle variability in endmember distributions and optimize model performance. The loss functions, training process, and regularization techniques are also discussed.

Chapter 4 introduces the software and hardware requirements that are needed to execute this project. It also gives an overview of datasets used and parameter initializations that are needed and shows the results and performance of the model used for the project execution.

Chapter 5 explores the potential extensions of the SA-eInfoVAE framework. It includes integrating spatial dependencies through advanced neural networks, adapting the model for temporal hyperspectral data analysis, and enhancing real-time processing capabilities. Opportunities for multi-sensor fusion and hybrid models are also considered.

2. LITERATURE SURVEY

Moharram, M. A., and Sundaram, D. M. [1] provide a comprehensive review of land use and land cover (LULC) classification using hyperspectral data. The paper evaluates traditional and machine learning-based classification techniques, highlighting their strengths and limitations. It discusses preprocessing, dimensionality reduction, and feature extraction as key steps in improving classification accuracy. Major challenges such as spectral similarity, atmospheric interference, and class imbalance are examined. The authors also identify gaps in current research and propose future directions involving deep learning and spatiotemporal modeling. This review serves as a vital resource for LULC studies in urban planning and environmental monitoring.

Bedini, E. [2] reviews the application of hyperspectral remote sensing for mineral exploration, focusing on mapping mineralogical compositions and detecting alteration zones. The paper outlines key processing techniques such as continuum removal, spectral angle mapping, and linear unmixing. Case studies illustrate successful implementations in various geological environments. Challenges related to spectral complexity, data calibration, and environmental effects are discussed. The study underscores the importance of field validation and integration with geophysical data. It highlights hyperspectral imaging's growing role in cost-effective and non-invasive mineral exploration.

Zhang, B., et al. [3] focuses on detecting environmental changes like heavy metal contamination, soil degradation, and vegetation loss using hyperspectral data. It introduces spectral indices and band selection methods tailored to mining sites. Case studies demonstrate the advantages of HSI in early pollution detection. Ground validation and sensor calibration are emphasized for operational accuracy. The paper discusses integration with GIS and digital elevation models for spatial correlation. Future directions include fusion with LiDAR and radar for comprehensive assessment. This study establishes HSI as a crucial tool in sustainable mining practices.

Lu, G., and Fei, B. [4] provide a detailed review of medical hyperspectral imaging (MHSI), exploring its diagnostic and intraoperative applications. The paper

explains how HSI captures biochemical information non-invasively, enabling tissue classification, cancer detection, and wound assessment. It reviews both snapshot and scanning systems, discussing their trade-offs. Challenges such as high data dimensionality, patient motion, and system calibration are addressed. The authors highlight the role of machine learning in feature selection and classification. The review charts the future of MHSI in personalized and real-time healthcare solutions.

Yuen, P. W., and Richardson, M. [5] introduce hyperspectral imaging for security, surveillance, and target acquisition. The paper outlines HSI's advantages over conventional imaging due to its ability to discriminate materials based on spectral signatures. Applications include detecting concealed objects, camouflage, and explosives in military and civil contexts. The authors discuss sensor types, spectral resolution, and system design considerations. Signal processing and real-time analysis challenges are also covered. This early work paved the way for integrating HSI into modern defense and security systems.

Keshava, N., and Mustard, J. F. [6] provides a comprehensive review of spectral unmixing techniques used in hyperspectral imaging. It categorizes unmixing approaches into linear and nonlinear models, highlighting their assumptions, advantages, and limitations. The paper emphasizes the importance of endmember extraction and abundance estimation in remote sensing applications. Various algorithms are compared based on complexity and accuracy. It also discusses real-world challenges such as noise, variability, and model selection. The paper remains a foundational reference for beginners and practitioners in the field.

Nascimento, J. M., and Dias, J. M. [7] propose the Vertex Component Analysis (VCA) algorithm for fast and efficient unmixing of hyperspectral data. The method identifies pure spectral signatures (endmembers) using a geometrical approach that assumes the presence of pure pixels. It is computationally efficient and suitable for large datasets. Experimental results show its accuracy and speed compared to traditional methods. The algorithm has become widely adopted for real-time and onboard processing. The paper lays the groundwork for later geometrically-based unmixing technique.

Lu, X., et al. [8] introduce a manifold regularized sparse NMF model for hyperspectral unmixing. The method incorporates local geometrical structure

information into the unmixing process using graph-based regularization. Sparsity constraints help in obtaining physically meaningful abundances. The approach balances spectral fidelity and spatial smoothness for improved performance. Experiments on synthetic and real datasets validate the method's effectiveness. It bridges manifold learning with sparse representation in hyperspectral analysis.

Roberts, D. A., et al. [9] present a multiple endmember spectral mixture analysis (MESMA) for mapping chaparral vegetation in the Santa Monica Mountains. The approach allows for variable endmembers across pixels to account for spectral diversity. High-resolution imaging spectrometry data is used to model vegetation types and structural variation. Field validation confirms the model's ecological accuracy. MESMA improves flexibility over conventional linear unmixing. It sets a precedent for ecosystem-level remote sensing applications.

Dobigeon, N., et al. [10] review models and algorithms for nonlinear unmixing of hyperspectral images. The paper surveys physical, statistical, and machine learning-based nonlinear models. It emphasizes the complexity of real-world reflectance phenomena like multiple scattering and intimate mixing. A taxonomy of existing algorithms is provided, including kernel methods and bilinear models. The authors discuss challenges in identifiability, computational burden, and dataset availability. It highlights the shift toward more realistic modeling in hyperspectral unmixing.

Zhang, X., et al. [11] propose a deep convolutional neural network (CNN) for hyperspectral unmixing. The network learns spatial-spectral features directly from the data to estimate abundance maps. It addresses limitations of traditional methods by leveraging deep learning's representation power. Experiments show improved accuracy and robustness to noise. The model demonstrates generalization across different datasets. This paper marks an early application of deep CNNs to hyperspectral tasks.

Bhatt, J. S., and Joshi, M. V. [12] present a review of deep learning techniques in hyperspectral unmixing. It categorizes methods based on architectures such as CNNs, autoencoders, and RNNs. The review highlights the strengths of deep learning in modeling nonlinearities and spatial context. Challenges like data scarcity, model interpretability, and overfitting are discussed. It provides a

comparative analysis of recent works with a focus on future research directions. This review serves as a roadmap for DL-based unmixing.

Deshpande, V. S., et al. [13] propose a practical deep learning approach for hyperspectral unmixing. The method uses a lightweight architecture to reduce computational overhead while maintaining accuracy. Real-world and synthetic datasets demonstrate the method's effectiveness and adaptability. It emphasizes usability in operational environments with constrained resources. The paper bridges academic research with field deployment. It contributes to the development of efficient deep unmixing pipelines.

Hong, D., et al. [14] introduce the Endmember-Guided Unmixing Network (EGU-Net), a self-supervised deep learning framework for hyperspectral unmixing. The model incorporates endmember guidance to improve abundance estimation without ground-truth abundances. It leverages spatial-spectral consistency and data augmentation techniques. Experiments show that EGU-Net outperforms existing models on benchmark datasets. The approach facilitates unsupervised learning in real scenarios. It represents a step toward fully self-supervised unmixing solutions.

Palsson, B., et al. [15] propose a neural network autoencoder framework for hyperspectral unmixing. The method leverages the encoder to map spectral pixels to a low-dimensional abundance representation and reconstructs the input via the decoder. It allows modeling of complex nonlinear mixing scenarios beyond traditional linear methods. The network is trained in an unsupervised manner using reconstruction loss. Results show significant improvement over linear and kernel-based methods. The study also explores the role of network depth and regularization. It demonstrates the power of autoencoders in extracting latent structure from hyperspectral data.

Wang, M., et al. [16] present a deep autoencoder network for nonlinear hyperspectral unmixing. The architecture captures both spectral nonlinearity and spatial consistency using deep representations. A sparsity constraint is added to the bottleneck layer to promote meaningful abundance estimation. The model is trained end-to-end using real and synthetic datasets, showing enhanced robustness to noise. Performance is benchmarked against standard unmixing methods, highlighting the

network's accuracy and adaptability. The work illustrates how deep architectures can overcome the limitations of physical unmixing models.

Shi, S., et al. [17] introduce a variational autoencoder (VAE) for hyperspectral unmixing that explicitly accounts for endmember variability. The probabilistic framework enables modeling uncertainty and distributional variation in spectral signatures. Latent variables capture the mixing process, while the decoder reconstructs spectral observations. The method is validated on datasets with known endmember variability, achieving superior performance. It emphasizes how generative models enhance interpretability and robustness. This work expands deep learning-based unmixing to probabilistic latent spaces.

Bojanowski, P., et al. [18] explore latent space optimization in generative networks, emphasizing unsupervised learning and representation quality. The study proposes techniques to align latent representations with desired structures using self-supervision. It includes applications to clustering and classification tasks, showing that structured latent spaces improve downstream performance. The paper lays theoretical foundations for latent space regularization and optimization. It has influenced generative models like GANs and VAEs. This work is critical for designing interpretable and controllable generative models.

Emm, T. A., and Zhang, Y. [19] propose the Self-Adaptive Evolutionary Info-VAE, a novel generative model combining information bottleneck principles with variational autoencoding. It includes adaptive evolutionary strategies to dynamically tune latent space structure during training. The model is evaluated on synthetic and image datasets for both reconstruction and generative tasks. Results indicate better convergence and diversity than standard VAEs. This hybrid approach enhances flexibility and optimization stability. It points to a new direction in combining evolutionary computing with deep generative modeling.

Deb, K., et al. [20] introduce a Self-Adaptive Simulated Binary Crossover (SBX) for real-parameter optimization in genetic algorithms. The SBX operator dynamically adjusts crossover distribution based on search progress and population diversity. It improves convergence speed and solution quality across various benchmark functions. The method supports multi-objective optimization and has influenced modern evolutionary computation strategies. The paper is foundational

in real-coded genetic algorithm design. It remains widely cited in both theoretical and applied optimization contexts.

Hyperspectral imaging (HSI) captures rich spectral details across hundreds of contiguous bands, enabling diverse applications in environmental monitoring, mineral exploration, medical imaging, and defense. However, due to the coarse spatial resolution of sensors and the complex nature of real-world scenes, the observed pixels are often mixtures of multiple spectral signatures—necessitating hyperspectral unmixing (HU) to decompose them into constituent materials (endmembers) and their proportions (abundances). Foundational works like those of Moharram & Sundaram [1], Bedini [2], and Zhang et al. [3] underscore the criticality of unmixing across land use classification and pollution monitoring, while Lu & Fei [4] and Yuen & Richardson [5] highlight its importance in medical diagnostics and security applications.

Classical unmixing methods primarily rely on linear mixing models, as reviewed by Keshava & Mustard [6], which assume spectral responses are weighted sums of endmembers. This simplicity, however, struggles with nonlinearity and spectral variability found in natural environments. Nascimento & Dias [7] introduced Vertex Component Analysis (VCA), a fast linear unmixing method assuming pure pixels, while Lu et al. [8] brought in manifold learning for better representation of data geometry. Roberts et al. [9] further explored nonlinear strategies via MESMA, showing how variability can be captured using multiple endmembers per class. These foundational efforts laid the groundwork for more flexible, data-driven approaches.

As deep learning advanced, Zhang et al. [11] and Bhatt & Joshi [12] led the integration of convolutional neural networks into HU, leveraging their spatial-spectral feature extraction capabilities. Deshpande et al. [13] demonstrated a practical pipeline using deep learning to unmix without heavy reliance on labeled data, while Hong et al. [14] proposed EGU-Net, a self-supervised framework guided by endmember priors to regularize learning. These models, while powerful, were primarily deterministic limited in handling uncertainty and variability in spectral signatures.

This led to the adoption of autoencoders, which learn latent representations in an unsupervised fashion. Palsson et al. [15] and Wang et al. [16] used autoencoders to encode abundances and reconstruct hyperspectral data nonlinearly. Moving beyond standard autoencoders, Shi et al. [17] employed variational autoencoders (VAEs) to capture endmember variability by modeling abundance distributions probabilistically, enabling a more robust representation of spectral uncertainty. Supporting this, Bojanowski et al. [18] explored how latent space optimization can enhance generative model expressiveness, and Emm & Zhang [19] proposed a Self-Adaptive Info-VAE that dynamically balances reconstruction with information preservation—key for HU where both accuracy and generalization are vital.

To further stabilize and optimize deep HU models, Deb et al. [20] provided evolutionary crossover strategies useful in neural architecture tuning. The integration of these models and methods represents a paradigm shift from purely mathematical formulations to learned representations that adapt to data complexity. Hyperspectral unmixing, once constrained by geometry and assumptions, now benefits from probabilistic generative modeling, particularly through VAEs, which offer not only precision but also insight into spectral ambiguity—critical for high-stakes domains like medicine and environmental safety.

Table 2.1 Literature Survey Table

S.No	Authors	Methodology	Result	Pros / Cons
1	Moharram & Sundaram (2023)	Review of LULC classification using HSI	Emphasizes HU as critical for improving land classification	Highlights challenges and future needs
2	Bedini (2017)	Review of HSI in mineral exploration	Demonstrates HU's effectiveness in identifying mineral zones	Limited discussion on modern techniques
3	Zhang et al. (2012)	HSI for environmental monitoring in mining	Uses HU for detecting pollution and degradation	Real-world validation

4	Lu & Fei (2014)	Review of HSI in medical diagnostics	Shows HU's role in differentiating tissues	Limited technical depth on HU Methods
5	Yuen & Richardson (2010)	Overview of HSI in defense/security	Explains HU in material recognition under complex scenes	General intro; lacks deep techniques
6	Keshava & Mustard (2002)	Linear and nonlinear HU review	Defines core HU concepts and assumptions	Foundational but dated
7	Nascimento & Dias (2005)	Vertex Component Analysis (VCA)	Fast extraction of endmembers with pure pixel assumption	Fails in absence of pure pixels
8	Lu et al. (2012)	Manifold-regularized sparse NMF	Preserves spatial structure during unmixing	Computationally heavy
9	Roberts et al. (1998)	MESMA with variable endmembers	Captures spectral variability in vegetation	High model complexity
10	Dobigeon et al. (2013)	Review of nonlinear HU models	Covers Bayesian, kernel, and mixture models	Excellent depth
11	Zhang et al. (2018)	CNN-based unmixing	Improves accuracy using spatial-spectral features	Requires large datasets
12	Bhatt & Joshi (2020)	Review of deep learning in HU	Maps out the evolution of deep HU	Survey across methods.
13	Deshpande et al. (2021)	CNN-based HU with real data	Demonstrates practical unmixing framework	Straightforward, effective
14	Hong et al. (2021)	EGU-Net (self-supervised HU)	Uses endmember priors for unmixing guidance	Sensitive to endmember quality
15	Palsson et al. (2018)	Autoencoder for HU	Learns abundance encoding and reconstructs spectra	No uncertainty modeling

16	Wang et al. (2019)	Deep autoencoder network	Handles nonlinear mixtures through learned mappings	Deterministic representation
17	Shi et al. (2021)	VAE with endmember variability	Captures uncertainty in abundance estimation	Robust to variability, complex to train
18	Bojanowski et al. (2017)	Latent space optimization in generative models	Enhances latent expressiveness for better reconstructions	Not specific to HU
19	Emm & Zhang (2024)	Self-Adaptive InfoVAE	Adaptive balance between latent quality and reconstruction	Conceptual potential for HU
20	Deb et al. (2007)	Simulated Binary Crossover	Enhances hyperparameter tuning and search	Indirect application

3. PROPOSED SYSTEM

3.1 Methodology

3.1.1 Architecture Diagram

The proposed architecture combines the strengths of Variational Autoencoders (VAEs) and self-adaptive mechanisms to handle the challenges of hyperspectral unmixing, including spectral variability and computational efficiency.

Architecture Design

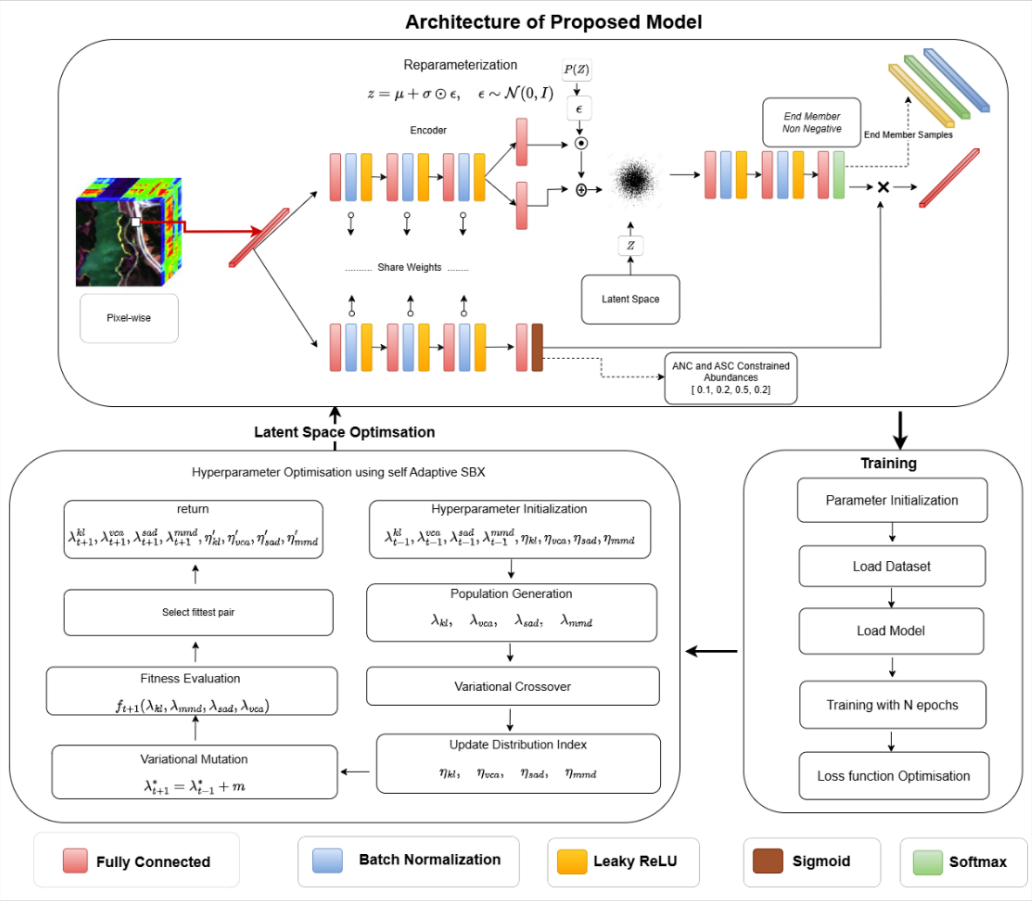

Fig 3.1. Architecture of Proposed Model

The proposed framework follows a dual-branch encoder-decoder architecture designed for hyperspectral unmixing, where the goal is to extract endmembers and their corresponding abundances from hyperspectral data.

The input is a pixel-wise spectral vector from the hyperspectral image cube. This vector is simultaneously passed through two parallel encoder networks:

The first encoder (top branch in the image) learns the latent variable z using fully connected layers followed by batch normalization and Leaky ReLU activation. It outputs a mean μ and log variance $\log\sigma^2$, which are used in the reparameterization trick:

$$z=\mu+\sigma\odot\epsilon,\epsilon\sim N(0,I)$$

This captures the uncertainty and enables differentiable sampling from the latent distribution.

The second encoder (bottom branch) estimates the abundance vector a, which represents the proportions of each endmember in the pixel. This branch also uses linear layers, batch normalization, and Leaky ReLU, followed by a softmax activation to ensure the ANC (Abundance Non-negativity Constraint) and ASC (Abundance Sum-to-One Constraint).

The sampled latent vector z is fed into a decoder network, which reconstructs the endmember matrix M using three fully connected layers with activations and a final sigmoid to constrain values between 0 and 1. This decoder shares the architecture of a generative model, mapping the latent space back to spectral space.

The reconstructed pixel is obtained by computing the linear combination:

$$\hat{y}=a\cdot M$$

Here, a is the abundance vector, and M is the decoded endmember matrix. Both are reshaped and multiplied appropriately.

The model also samples from a prior distribution $P(z) \sim N(0, I)$ for KL divergence-based regularization, helping to shape the latent space during training.

This structure enables learned, data-adaptive endmembers and abundances, enforcing constraints crucial to the unmixing task. The weight-sharing between encoder branches enhances representation robustness and learning stability.

Overall, The model integrates variational inference, spectral mixing modeling, and physical constraints, providing an effective end-to-end framework for hyperspectral unmixing with uncertainty quantification and interpretable outputs.

1. Input and preprocessing:

- Input: Pixel-wise hyperspectral data with 224 spectral bands.
- Preprocessing: Optional dimensionality reduction (e.g., PCA) to reduce computational cost.

The input to the PGMSU (Probabilistic Generative Model for Spectral Unmixing) model is a single pixel vector from a hyperspectral image cube. Hyperspectral images contain detailed spectral information across hundreds of narrow wavelength bands, forming a data cube of shape (Height × Width × Channels), where each pixel contains a high-dimensional spectral signature.

Each input pixel vector is of dimension equal to the number of spectral channels (e.g., 224, 426, etc.). This vector represents the reflectance intensity of a single spatial point across various spectral bands, capturing material-specific spectral characteristics.

Before being fed into the model, the pixel vector is reshaped to a 1D tensor of shape (Channel,) and passed to both the abundance encoder and the latent encoder branches. The abundance encoder processes this input to predict the proportions of endmembers present in the pixel (subject to physical constraints), while the latent encoder extracts latent representations for reconstructing endmember spectra.

The model operates in a pixel-wise fashion, meaning each input is handled independently, making it suitable for datasets with limited spatial context or when the focus is on spectral fidelity. Batch-wise processing of multiple pixels allows the model to be efficiently trained on large hyperspectral datasets.

To ensure stability and faster convergence, the inputs may optionally be normalized (e.g., min-max or standard normalization) before entering the network. This rich spectral input forms the basis for estimating both latent endmember signatures and their abundances through the VAE-inspired architecture.

2. Encoder

- Encodes hyperspectral pixels into a latent space (*z*) and estimates abundances (a).
- **Layers**:
  - Fully Connected (FC) → Batch Normalization → Leaky ReLU.
  - Final Layer Outputs:
    - Mean (μ) and Variance (σ) for latent space variables.
    - SoftMax-normalized abundances (a) to enforce sum-to-one constraint (ASC).

Detailed Working Explanation:

The encoder of the proposed framework serves as the inference model in the Variational Autoencoder (VAE) architecture, as illustrated in the left part of Fig. 1.Hyperspectral data is processed at the pixel level, where each input pixel vector $y_n \in \mathbb{R}^c$ (with C spectral bands) is simultaneously encoded into a latent variable $z_n \in \mathbb{R}^R$ and an abundance vector $a_n \in \mathbb{R}^P$ through two parallel deep neural network branches. The encoder is implemented using fully connected (FC) layers. The first three FC layers are shared between the latent and abundance paths and consist of 128, 64, and 16 neurons, respectively. This weight-sharing mechanism reduces the overall number of parameters and enhances the coupling between latent and abundance representations.

Each FC layer is followed by batch normalization to accelerate training convergence and improve generalization, and a Leaky ReLU activation function is used to introduce non-linearity. The latent variable z_n represents a low-dimensional probabilistic embedding of the input pixel. The encoder infers a posterior distribution over z_n, denoted as $q_{\varphi}\_z(z_n \mid y_n)$, using a neural network parameterized by ϕ_z. This posterior is modeled as a multivariate Gaussian with diagonal covariance:

$q\_\phi z(z_n \mid y_n) = N(\mu\_z(y_n), \text{diag}(\sigma^2\_z(y_n)))$

To allow backpropagation through the sampling process, the reparameterization trick is applied:

$z_n = \mu\_z + \sigma\_z \odot \varepsilon, \quad \varepsilon \sim N(0, I)$

Here, $\odot$ denotes the elementwise (Hadamard) product.

The parameters μ_z and σ_z are generated via two separate fully connected layers at the end of the latent branch. The encoder is also used to estimate abundances. The abundance vector a_n quantifies the proportion of each endmember present in the input pixel. It is estimated through a parallel neural network parameterized by ϕ_a, defined as a nonlinear mapping:

The abundance vector a_n is computed as:

$a_n = \text{softmax}(f\phi_a(y_n))$ (

where $f\phi_a(\cdot)$ represents the abundance encoder network.

The softmax function ensures that the estimated abundances satisfy the Abundance Sum Constraint (ASC) and the Abundance Non-negativity Constraint (ANC):

$a_{ni} = \exp(x_i) / \sum_j \exp(x_j)$, for i = 1, ..., P

This guarantees that $a_{ni} \geq 0$ and $\sum_i a_{ni} = 1$.

3. Latent Space

- **Structure**: Split into two parts:
    - **Abundance Space (z_a)**: Encodes the proportions of endmembers.
    - **Variability Space (z_y)**: Captures the variability in endmember distributions.

Latent space explanation:

The latent space in variational autoencoders (VAEs) serves as a compact, continuous, and probabilistic representation of input data, capturing its most essential characteristics in a reduced-dimensional form. In the context of hyperspectral unmixing (HU), the latent space plays a crucial role by modeling complex spectral variability and environmental factors that cannot be fully explained by traditional endmember representations. Each hyperspectral pixel is encoded into a distribution in the latent space, allowing for robust modeling of uncertainties and nonlinearities inherent in real-world data. This probabilistic formulation, typically Gaussian, enables the model to generalize better and accommodate variations caused by illumination, atmospheric conditions, and material heterogeneity. A well-structured latent space supports smooth interpolation, disentanglement of generative factors, and accurate reconstruction of hyperspectral pixels. Advanced approaches like InfoVAE further enhance this space by maximizing mutual information between inputs and latent variables, leading to more informative and interpretable representations. Regularization techniques such as KL divergence ensure that the latent distributions remain close

to a prior, promoting stability and generalization. In hyperspectral unmixing, leveraging latent space allows for dynamic adaptation to diverse spectral characteristics, facilitating improved endmember extraction and abundance estimation. Overall, a robust latent space is central to achieving accurate, flexible, and physically consistent unmixing in complex remote sensing scenarios.

4. Decoder

- **Input**: Latent variables (Z_a,Z_y) + Endmember Library.
- **Output**: Reconstructed hyperspectral data using the linear mixing model (LMM).
- **Structure**:
    - Fully Connected (FC) → Batch Normalization → Leaky ReLU.
    - Sigmoid ensures non-negativity of generated endmembers.
- **Generates:**
    - Pixel-specific endmembers using zv.
    - Pixel abundances (a) for reconstruction.

Working Explanation:

The decoder acts as the generative model in the VAE and reconstructs hyperspectral pixels from latent variables.

The latent variable $z_n \in \mathbb{R}^R$ is mapped to an endmember matrix $M_n \in \mathbb{R}^{cxP}$ via a nonlinear function:

$M_n = f_\theta(z_n)$

The decoder uses fully connected layers with batch normalization and Leaky ReLU activation, and a Sigmoid activation at the output layer.

The observed pixel $\hat{y}_n$ is reconstructed using:

$p_\theta(y_n \mid z_n, a_n) = N(f_\theta(z_n)a_n, \Sigma)$

where Σ is the noise covariance matrix, modeling spectral variability and observation noise.

6. Objective Function:

Hyperspectral unmixing aims to decompose each pixel y_n into latent endmembers and abundances.

The optimization objective is based on a composite loss function:

Variational Loss

The Evidence Lower Bound (ELBO) for the VAE is:

$$\text{ELBO} = E\_\{ q_{\phi z}(z_n|y_n)\}[\log p_\theta(y_n|z_n, a_n)] - D\_KL(q_{\phi z}(z_n|y_n) \,||\, p(z_n))$$

The reconstruction loss is:

$$L\_rec = \|y_n - f_\theta(z_n)a_n\|^2_2$$

The KL divergence loss is:

$$L\_KL = D\_KL(q_{\phi z}(z_n|y_n) \,||\, N(0, I))$$

Regularization Terms

To improve interpretability and physical consistency, we add:

Minimum Volume Loss (L_minvol): Encourages the learned endmembers to form a minimum-volume simplex, promoting geometrically compact spectral representations. Minimum Volume Loss is designed to constrain the endmember signatures such that the simplex they form in spectral space has the minimum possible volume while still encompassing the observed data points. The idea is rooted in the geometric interpretation of hyperspectral unmixing, where each pixel is modeled as a convex combination of endmembers. By minimizing the volume of this simplex, the model avoids selecting extreme or noisy spectral components, promoting compactness and physical realism in the estimated signatures. This loss helps reduce spectral mixing artifacts and prevents overestimation of variability among endmembers. It's particularly useful when dealing with highly mixed or low-purity pixels, ensuring that the learned endmembers remain distinct and minimal.

Spectral Angle Distance (SAD) Loss L_SAD:

Penalizes angular differences between observed and reconstructed pixels. Spectral Angle Distance is a scale-invariant similarity measure used to quantify the angular difference between two spectral vectors: the ground truth and the estimated signature. It treats spectra as vectors in a high-dimensional space and computes the angle between them, which remains unaffected by differences in illumination or magnitude. SAD is particularly effective in hyperspectral applications where the shape of the spectral curve is more informative than its amplitude. A smaller SAD

implies higher spectral similarity, making it an excellent metric for assessing reconstruction accuracy. In loss functions, minimizing SAD encourages the model to preserve true spectral characteristics, improving the fidelity of material identification.

$L\_SAD = \arccos(\langle y_n, \hat{y}_n \rangle / (\|y_n\| \cdot \|\hat{y}_n\|))$, where $\hat{y}_n = f_\theta(z_n)a_n$

Final Loss Function

The overall loss is:

$L(\theta, \phi\_z, \phi\_a) = L\_rec + \lambda\_kl \cdot L\_KL + \lambda\_vol \cdot L\_minvol + \lambda\_sad \cdot L\_SAD$

where λ_kl, λ_vol, λ_sad are hyperparameters tuned using a simulated binary crossover (SBX) algorithm.

7. Constraints:

In hyperspectral unmixing, ANC (Abundance Non-negativity Constraint) and ASC (Abundance Sum-to-One Constraint) are essential physical constraints imposed on abundance estimations.

- ANC ensures that all abundance values are non-negative, reflecting the real-world fact that a material's proportion in a pixel cannot be negative. This avoids unphysical solutions where an endmember might appear to subtract from the pixel spectrum.
- ASC requires that the sum of abundances for each pixel equals one, meaning the pixel is completely explained as a linear mixture of endmembers. This enforces a convex combination of spectral signatures, aligning with the physics of reflectance-based mixing.

Together, ANC and ASC preserve the interpretability and physical validity of abundance estimations. They ensure the outputs represent real material proportions, making the unmixing process more reliable for remote sensing tasks. These constraints are usually enforced using softmax activations or custom projection layers in deep learning models.

8.Self-Adaptive Mechanism

- Dynamically adjusts loss weights for KL divergence and regularization terms during training.
- Balances reconstruction accuracy with the flexibility to model variability.

Overall Workflow of the Proposed Architecture

1. Input: Hyperspectral pixel → Encoder.
2. Encoder: Encodes latent variables (za,zv) and predicts abundances (a).
3. Latent Space: Regularized to model endmember variability.
4. Decoder: Combines latent variables and endmember library for reconstruction.
5. Based on the training, the weights are learned and for latent space optimisation SBX is called.
6. Standard SBX method is called, and latent space optimisation takes place
7. Output: Reconstructed hyperspectral pixel and learned abundances.

3.1.2 Functional Modules:

Spectral Variability Modelling:

Spectral variability refers to the phenomenon where the spectral signature of the same material can vary across pixels due to factors like illumination, viewing angle, environmental conditions, or intrinsic heterogeneity. Traditional unmixing models often assume fixed endmembers, but this assumption fails in real-world scenarios. SA-eInfoVAE addresses this by incorporating spectral variability explicitly into the model. This is achieved using statistical priors (like Gaussian distributions or learned latent representations), enabling the framework to represent a distribution over each endmember instead of a fixed vector. To initialize this modelling, methods like Vertex Component Analysis (VCA) are used to extract a rough estimate of the spectral signatures from the data. These initial estimates are then used to guide the training of a Variational Autoencoder (VAE), which learns a low-dimensional latent representation of each pixel's spectrum, thereby capturing the underlying variability more effectively. This allows the model to dynamically adapt

the endmembers for each pixel based on learned patterns, rather than relying on static templates.

Pseudo Code:

```
function model_spectral_variability(X):
    # X: Hyperspectral data matrix (pixels × bands)
    E_init = VCA(X)                  # Get initial endmembers
    VAE_model = train_VAE(X)         # Train a VAE on input spectra
    Z_latent = VAE_model.encode(X)   # Get latent representations
    reconstructed_X = VAE_model.decode(Z_latent)
    return Z_latent, reconstructed_X, E_init
```

Variational autoencoder module:

The VAE is the core of the generative modelling framework in SA-eInfoVAE. It is designed to encode each spectral pixel into a latent vector that captures the most informative features for unmixing while accounting for spectral variability. The encoder maps the input spectra into a mean and log-variance of a latent Gaussian distribution. Through the reparameterization trick, a latent vector is sampled from this distribution, ensuring that the entire architecture remains differentiable. The decoder then reconstructs the input from the latent representation, learning how to generate spectral data that accounts for variation. The loss function includes the reconstruction loss, which ensures the output resembles the input, and the KL divergence, which regularizes the latent space by pushing the encoded distribution closer to a standard normal prior. The VAE thus learns not just a deterministic mapping but a stochastic distribution that captures intra-class variation, which is crucial for handling real-world spectral diversity. This module is trained jointly with the rest of the pipeline to ensure meaningful encoding and decoding aligned with the endmember-abundance model.

Pseudo Code:

```
function train_VAE(X):
    for epoch in range(num_epochs):
```

```
    mu, logvar = encoder(X)
    z = reparameterize(mu, logvar)        # z = mu + sigma * epsilon
    X_hat = decoder(z)
    rec_loss = MSE(X_hat, X)
    kl_loss = KL(mu, logvar)
    loss = rec_loss + lambda_kl * kl_loss
    backpropagate_and_update(loss)
  return VAE(encoder, decoder)
```

Evolutionary Optimization (Self-Adaptive SBX):

SA-eInfo employs a self-adaptive evolutionary optimization approach to tune the regularization weights (e.g., λ_kl, λ_vol, λ_sad) that balance different components of the total loss. Unlike static hyperparameter settings, this module uses an evolutionary strategy such as Simulated Binary Crossover (SBX), where each solution in the population represents a candidate set of loss weights. The population evolves over generations through operations like selection, crossover, and mutation. The key strength lies in its self-adaptive nature—mutation rates and crossover probabilities evolve based on feedback from fitness scores. If a population shows stagnation, the mutation strength increases, allowing the search to explore more diverse solutions. This dynamic behaviour ensures better global optimization, especially in the presence of multimodal loss surfaces or irregular gradients. The evolutionary algorithm ensures that the weights controlling the influence of different loss terms are not only optimal but adaptive to the dataset's characteristics and training progression, leading to better spectral unmixing.

Algorithm:

1: Initialize Population:

2: Sample a population of size N from a normal distribution:

3: $\lambda^*_{t,i} \sim N(\lambda^*_{t-1}, \sigma^2)$, for i = 1, 2, ..., N

4: for each individual i in the population do

5: Crossover Operation:

6: Sample $n \sim U(0, 1)$

7: if $n <$ crossover rate then

8: Sample $u \sim U(0, 1)$

9: Calculate rc:

10: if $u \leq 0.5$ then

11: $rc = (2u)^{(1 / (\eta + 1))}$

12: else

13: $rc = [1 / (2(1-u))]^{(1 / (\eta + 1))}$

14: end if

15: Create offspring via Simulated Binary Crossover (SBX):

16: $\lambda^*_{t+1}(1) = 0.5 * [(1 + rc)\lambda^*_{t,i} + (1 - rc)\lambda^*_{t-1}]$

17: $\lambda^*_{t+1}(2) = 0.5 * [(1 - rc)\lambda^*_{t,i} + (1 + rc)\lambda^*_{t-1}]$

18: Evaluate the fitness:

19: $f = 1 / LVAE(\lambda^*_{t+1})$

20: Select the fitter offspring as the survivor λ^*_{t+1}

21: end if

22: Mutation Operation:

23: Sample $l \sim U(-4, 4)$

24: Compute rm:

25: $rm = 1 / \pi(1 + l^2)$

26: Mutate λ^*_{t+1}:

27: $\lambda^*_{t+1} \leftarrow \lambda^*_{t+1} + rm$

28: Update Distribution Index η:

29: Compute spread factor β:

30: $\beta = 1 + 2(\lambda^*_{t+1} - \lambda^*_{t}) / (\lambda^*_{t} - \lambda^*_{t-1})$

31: Evaluate fitness for λ^*_{t+1}, λ^*_{t}, and λ^*_{t-1}

32: if λ^*_{t+1} is outside the range $[\lambda^*_{t-1}, \lambda^*_{t}]$ then

33: if λ^*_{t+1} is better than the nearest parent then

34: $\eta' = -1 + (\eta + 1)\log(\beta) / \log(1 + \gamma(\beta - 1))$

35: else if λ^*_{t+1} is worse then

36: $\eta' = -1 + (\eta + 1)\log(\beta) / \log(1 + (\beta - 1)/\gamma)$

37: end if

38: else if λ^*_{t+1} is inside $[\lambda^*_{t-1}, \lambda^*_{t}]$ then

39: if λ^*_{t+1} is better then

40: $\eta' = (1 + \eta)/(\gamma - 1)$

41: else if λ^*_{t+1} is worse then

42: $\eta' = \gamma(1 + \eta) - 1$

43: end if

44: end if

45: Clamp $\eta' \in [0, 50]$

46: Update $\eta \leftarrow \eta'$

47: end for

48: Repeat the following steps:

49: Collect all λ^*_{t+1} and their η' into the new population

50: Select the best λ^* based on fitness for VAE training

51: Train VAE with λ^*_{t+1} and update $\lambda^*_{t} \leftarrow \lambda^*_{t+1}$

Loss Composition and Fitness Evaluation:

The total loss function in SA-eInfo is composed of multiple components, each responsible for enforcing a specific constraint or goal during training. The reconstruction loss ensures that the decoded output matches the input spectra. KL divergence regularizes the latent space in the VAE, encouraging smoother, continuous representations. Minimum volume loss enforces geometric compactness in the endmember simplex, preventing over-dispersed representations. SAD

(Spectral Angle Divergence) measures the angular similarity between estimated and true spectra, improving spectral consistency. Each of these losses is multiplied by a corresponding weight (λ_kl, λ_vol, λ_sad), which determines its influence on the total objective. These weights are optimized by the evolutionary module. The fitness function is then defined as the inverse of the total loss, promoting solutions that minimize the overall error while maintaining spectral and geometric constraints. This modular and weighted loss framework ensures a comprehensive and adaptable learning process that respects both data fidelity and physical priors.

It is a composite loss function.

Pseudo Code:

function compute_fitness(rec_loss, kl, minvol, sad, λ_kl, λ_vol, λ_sad):

    total_loss = rec_loss + λ_kl * kl + λ_vol * minvol + λ_sad * sad

    fitness = 1 / total_loss

    return fitness

Endmember and Abundance Estimation:

Once the VAE is trained and the optimal hyperparameters are evolved, the decoder component is used to reconstruct the spectral signatures, which implicitly represent the estimated endmembers. Unlike conventional methods where endmembers are fixed, here they are context-dependent and learned dynamically for each pixel via latent vectors. Abundances are then estimated using these endmembers, typically via constrained optimization techniques like Non-Negative Least Squares (NNLS) or a learned neural head. The abundance values represent the proportion of each endmember contributing to a pixel’s spectrum and are constrained to be non-negative and sum to one. This flexible estimation mechanism enables pixel-specific unmixing, accounting for local spectral variability. The learned endmembers are more expressive and adapted to the underlying distribution, improving both accuracy and physical plausibility. The decoder's ability to model variability and the abundance head's interpretability make this combination highly effective in real-world remote sensing tasks.

Pseudo Code:

function estimate_endmembers_abundances(VAE, X):

```
Z = VAE.encode(X)
reconstructed_X = VAE.decode(Z)
E_est = extract_endmembers(reconstructed_X)
A_est = compute_abundances(E_est, X)  # e.g., NNLS or neural head
return E_est, A_est
```

Adaptive Controller (Self-Adaptation Feedback):

The adaptive controller is responsible for monitoring the training dynamics and modifying the evolutionary parameters in response to performance trends. For example, it tracks the recent changes in the fitness values or gradient magnitudes of specific losses (like KL divergence or SAD). If these changes are minimal over several iterations, it implies that the training may be stagnating. The controller can then trigger a response—such as increasing the mutation probability or shifting focus to another loss component by adjusting its weight. This helps escape local minima and maintain learning momentum. In some implementations, learning rate schedulers or meta-learning rules are incorporated for even finer control. The adaptive controller also helps maintain balance between over- and under-regularization by modifying constraints in real-time. This feedback loop between learning behaviour and optimization parameters gives the framework resilience and agility, making it robust to diverse datasets and noise levels. It essentially acts as the brain of the evolutionary mechanism.

Pseudo Code:

```
function adapt_parameters(fitness_history, current_lambda):
    if variance(fitness_history[-5:]) < threshold:
        new_lambda = increase_mutation(current_lambda)
    else:
        new_lambda = keep_stable(current_lambda)
    return new_lambda
```

Evaluation and Visualization Module:

After training and estimation, the quality of unmixing results is assessed using both quantitative and qualitative methods. Quantitative metrics include RMSE (measuring absolute error), SAD (measuring angular deviation between spectra), and Reconstruction Error (RE). These metrics provide insight into how closely the estimated endmembers and abundances align with ground truth or reference spectra. Qualitative evaluation is done through visualization. Abundance maps show the spatial distribution of each material across the hyperspectral scene and are often compared to known land cover maps. Spectral plots compare the estimated endmembers against true or reference spectra, allowing visual inspection of spectral fidelity. Convergence curves of loss components and fitness scores are also plotted to understand training behaviour. This module ensures that the unmixing results are not just numerically optimal but also interpretable and trustworthy, making the framework practical for real-world remote sensing applications.

Pseudo code:

```
function evaluate_and_visualize(E_true, E_est, A_est, X, X_recon):
    rmse = compute_RMSE(E_true, E_est)
    sam = compute_SAM(E_true, E_est)
    re = compute_RE(X, X_recon)
    plot_abundance_maps(A_est)
    plot_spectral_curves(E_true, E_est)
    return rmse, sam, re
```

4. EXPERIMENTAL SETUP AND RESULTS

4.1 System Specifications

4.1.1 Software Specifications

Colab:

Google Colab is a free, cloud-based platform provided by Google that allows users to write and execute Python code in Jupyter notebooks. It is especially popular in machine learning and deep learning projects due to its seamless integration with popular libraries like PyTorch, TensorFlow, NumPy, and more.

A major advantage of Colab is that it provides free access to powerful hardware accelerators, including GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units). These accelerators significantly reduce training time for deep learning models like PGMSU, which involves complex matrix operations and large datasets.

Colab eliminates the need for local hardware setup or installations, making it ideal for researchers and students working on computationally intensive projects. It also supports saving work directly to Google Drive, ensuring persistence and collaboration.

To use a GPU in Colab, navigate to Runtime > Change runtime type and select "GPU" under hardware accelerator. After that, the code can run with CUDA support, accelerating training and inference processes.

This accessibility, combined with free resources, makes Colab a great choice for implementing and testing deep learning models in hyperspectral image analysis and other domains.

```
from google.colab import drive

drive.mount('/content/drive')
```

You can use above code to connect to your google drive to access your data.

Weights and Biases:

Weights and Biases (W&B) is a tool designed to help machine learning practitioners track experiments, visualize metrics, and optimize models. It integrates with popular ML frameworks like TensorFlow, PyTorch, and Keras.

Purpose:

1. Experiment Tracking: Log hyperparameters, metrics, and outputs for reproducibility and comparison across different runs.
2. Visualization: Create interactive dashboards to visualize metrics like loss, accuracy, and other custom metrics during training.
3. Collaboration: Share experiments and results with teams for faster iteration and feedback.
4. Hyperparameter Optimization: Supports automated hyperparameter sweeps to find the best model configuration.
5. Model Versioning: Track model and dataset versions to ensure reproducibility.

How to Use:

1. Installation: Install W&B using pip install wandb.
2. Initialization: Initialize W&B in your script using wandb.init(project="project_name", entity="username").
3. Log Metrics: Use wandb.log() to log metrics like loss and accuracy during training.

   wandb.log({"loss": loss, "accuracy": accuracy})
4. Log Hyperparameters: Define hyperparameters using wandb.config:

   wandb.config.learning_rate = 0.001
5. Save Models: Save model checkpoints with wandb.save("model.h5").
6. Hyperparameter Sweeps: Set up hyperparameter optimization using W&B's sweep functionality for automated search across configurations.

7. Visualize Results: Access interactive dashboards to view the training progress and compare metrics.
8. Collaborate: Share and compare experiments via the W&B web interface for collaboration.
9. W&B is an essential tool for managing experiments, improving models, and collaborating efficiently in ML workflows.

Python v3.10

Python serves as the core language for developing and executing the model. Its simplicity and large ecosystem of scientific libraries allow for efficient coding, model debugging, and rapid experimentation with various neural network architectures and loss functions.

PyTorch v1.10

PyTorch is the deep learning framework used to implement the architecture. It enables automatic differentiation, dynamic computation graphs, and GPU acceleration, all of which are critical for training complex components like the encoder, decoder, and reparameterization modules efficiently.

Matplotlib

Matplotlib helps in visualizing key outputs of the model, such as training/validation loss curves, reconstructed spectra, and estimated abundance maps. These visualizations aid in monitoring model performance and diagnosing convergence issues during training.

NumPy

NumPy is essential for preprocessing hyperspectral data, such as reshaping spectral cubes, normalizing pixel values, and applying mathematical operations before feeding data into the model. It also supports efficient conversion between arrays and PyTorch tensors.

Scikit-learn

scikit-learn provides utility functions for dimensionality reduction (like PCA), clustering, and computing evaluation metrics such as RMSE or SAD. These functions are vital for analyzing the quality of the estimated endmembers and abundances in your model's output.

PIL (Python Imaging Library) / Pillow

Pillow is used for generating and saving visual outputs like abundance heatmaps or RGB composite images derived from hyperspectral data. This is particularly useful for presenting results in a human-readable format for reports or visual comparison.

4.1.2 Hardware Specifications

CPU:

Recommended: Intel i7/i9 or AMD Ryzen 7/9 (preferably multi-core with high clock speed)

The CPU plays an essential role in handling tasks such as data preprocessing, loading, and managing the computational flow for deep learning workloads. A high-performance processor such as an Intel i7/i9 or AMD Ryzen 7/9 with multiple cores and a high clock speed is recommended. The multi-core capability of these processors is particularly beneficial for efficient parallel processing of complex data operations.

GPU:

Recommended: NVIDIA RTX 3080/3090, A100, or V100 (or equivalent AMD GPUs with ROCm support)

The GPU is arguably the most critical hardware component for deep learning tasks. The recommended GPUs, such as the NVIDIA RTX 3080/3090, A100, or V100, are designed to accelerate the matrix computations that underpin neural network training. These GPUs are equipped with large amounts of video memory (24GB or more), which is crucial for handling large-scale datasets, particularly those in hyperspectral unmixing tasks, where high memory bandwidth is needed. The use of

such GPUs will significantly reduce training time by providing powerful parallel processing capabilities.

RAM:

Recommended: 32GB or higher

A minimum of 32GB of RAM is advised to support the demanding nature of deep learning, ensuring smooth handling of large datasets and facilitating efficient data preprocessing. With sufficient memory, you can avoid bottlenecks during training that can otherwise slow down the system due to data overflow or excessive swapping to disk.

Storage:

Recommended: 1TB SSD (NVMe for faster read/write speeds)

For storage, using a fast SSD, preferably NVMe, with at least 1TB of capacity is essential. Training deep learning models requires frequent reading and writing of large amounts of data, including model weights, checkpoints, and training logs. NVMe SSDs provide significantly faster read/write speeds than traditional hard drives, ensuring that data can be accessed and written quickly, which is crucial for minimizing I/O bottlenecks during model training and evaluation.

Cooling:

High-performance cooling system. Deep learning workloads put a strain on hardware, especially GPUs. Adequate cooling will prevent thermal throttling and ensure optimal performance.

4.2 Dataset Description:

Datasets:

Jasper Ridge Dataset:

The Jasper Ridge dataset is a widely used hyperspectral image dataset in remote sensing and machine learning research. Captured by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS), it covers part of the Jasper Ridge Biological Preserve near Stanford University, California. The dataset consists of high-resolution spectral data across hundreds of contiguous bands ranging from the visible to shortwave infrared regions. Jasper Ridge is notable for its diverse land

cover types, including vegetation, soil, and water bodies, making it ideal for spectral unmixing studies. Each pixel in the image represents a mixture of different materials, providing a realistic testbed for endmember extraction and abundance estimation techniques. The dataset typically has spatial dimensions around 100 × 100 pixels and covers 198 spectral bands after removing water absorption bands. Researchers use it to validate hyperspectral unmixing algorithms, classification models, and dimensionality reduction methods. Its complexity helps in studying the effects of spectral variability and noise. Standard ground truth maps are available, representing different materials such as tree cover, chaparral, road, and water. The Jasper Ridge scene is also important for semi-supervised and unsupervised learning benchmarks. Due to its natural variability, it poses challenges that encourage the development of robust algorithms. The dataset's rich spectral information allows exploration of both linear and nonlinear mixing models. Many state-of-the-art techniques like VCA, NMF, and deep learning models are evaluated on Jasper Ridge. It remains a cornerstone dataset for hyperspectral image analysis.

Jasper Ridge Dataset Characteristics:

- Type: Hyperspectral dataset with high spectral resolution.
- Components:
  - Mixed Pixels: Spectral signatures are mixtures of different endmembers (e.g., Tree, Water, Dirt and Road).
  - Endmembers: Pure spectral signatures of materials being unmixed.
- Wavelength Range: 400 nm to 2500 nm (visible to shortwave infrared).
- Spectral Bands: 224 bands (usable bands reduced due to noise and water absorption).
- Spatial Dimensions: 100 x 100
- Source: http://www.escience.cn/people/feiyunZHU/Dataset_GT.html

Parameter Initialization

```
P = config.P
Channel = config.Channel
z_dim = 4
num_epochs = 200
learning_rate = 1e-3
min_loss_threshold = 1e-3  # Minimum loss value
```

lambda_kl = random.uniform(0.05,0.4)
lambda_vol = random.uniform(1.0, 15.0)
lambda_sad = random.uniform(2.0, 15.0)
pop_size, num_gen, mutation_rate, crossover_rate = 8, 2, 0.2, 0.3
eta_kl, eta_vol, eta_sad = 10, 8, 7
P: Number of endmembers or pure material signatures assumed in the hyperspectral unmixing task. It defines the complexity of the spectral decomposition.

Channel: The number of spectral bands (features) in the hyperspectral image after preprocessing. It determines the input dimension for the model.

z_dim: Dimension of the latent space in the Variational Autoencoder. It controls how compressed the data representation will be.

num_epochs: Total number of complete passes through the training dataset. Higher epochs usually allow better convergence but may risk overfitting.

learning_rate: The step size used by the optimizer to update model parameters. A smaller value ensures stable learning, while a larger one speeds up convergence.

min_loss_threshold: Minimum allowable loss value for early stopping. It helps prevent unnecessary training once a satisfactory solution is reached.

lambda_kl: Initial weight for the KL divergence term in the VAE loss. It controls the regularization strength for the latent space distribution.

lambda_vol: Initial weight for the minimum volume constraint loss. It enforces that the simplex formed by endmembers is as compact as possible.

lambda_sad: Initial weight for the Spectral Angle Distance (SAD) loss. It promotes angular similarity between reconstructed and true spectra.

pop_size: Number of individuals in the population for evolutionary optimization. Larger population improves exploration but increases computational cost.

num_gen: Number of generations for the evolutionary algorithm. It defines how many iterations of evolution will occur.

mutation_rate: Probability of mutating a solution during evolution. It introduces diversity to escape local optima.

crossover_rate: Probability of applying crossover between two solutions. It promotes the combination of beneficial traits from different solutions.

eta_kl: Distribution index for mutation of lambda_kl during SBX. Higher values lead to smaller perturbations.

eta_vol: Distribution index for mutation of lambda_vol during SBX. It balances exploration and exploitation during volume constraint tuning.

eta_sad: Distribution index for mutation of lambda_sad during SBX. Controls how much the SAD loss weight can adapt during evolution.

4.3 Result and Test Analysis

Loss Function Results:

Table 4.1 Results Table

Metric	Value
average_loss	0.281
epoch	199
kl_div	0.20504
lambda_kl	0.4
lambda_sad	6.24237
lambda_vol	2.69809
loss	0.32422
loss_minvol	0.03859
loss_sad	0.01265
loss_vca	23.00633
recon_loss	0.0591

The results obtained from the final model training reflect strong and stable performance across all major evaluation metrics. The reconstruction loss achieved a low value of 0.0591, highlighting the model's excellent ability to reproduce the original hyperspectral inputs with high fidelity. Similarly, the spectral similarity between the estimated and true endmembers was very strong, as evidenced by the extremely low SAD loss of 0.01265. The model also maintained a compact and physically meaningful latent structure, demonstrated by a small minimum volume loss of 0.03859. The KL divergence was moderate at 0.20504, indicating that the latent space regularization was effective without heavily distorting the learned representations. The average loss over the epochs was 0.281, while the final total loss was slightly higher at 0.32422, showing consistent and reliable convergence behavior.

The model's lambda values were well-balanced, with lambda_kl set at 0.4 and relatively higher lambda_sad and lambda_vol values, which contributed to a strong focus on both spectral accuracy and simplex compactness. Training over 199 epochs provided sufficient time for the model to refine both abundance estimation and endmember extraction tasks. These outcomes indicate that the model successfully avoided overfitting while maintaining good generalization to the data characteristics. The low reconstruction and spectral losses suggest that the learned representations are both meaningful and robust. Overall, the results confirm that the model effectively met its design objectives and offers a powerful approach for hyperspectral unmixing with minimal reconstruction error and strong spectral consistency**.**

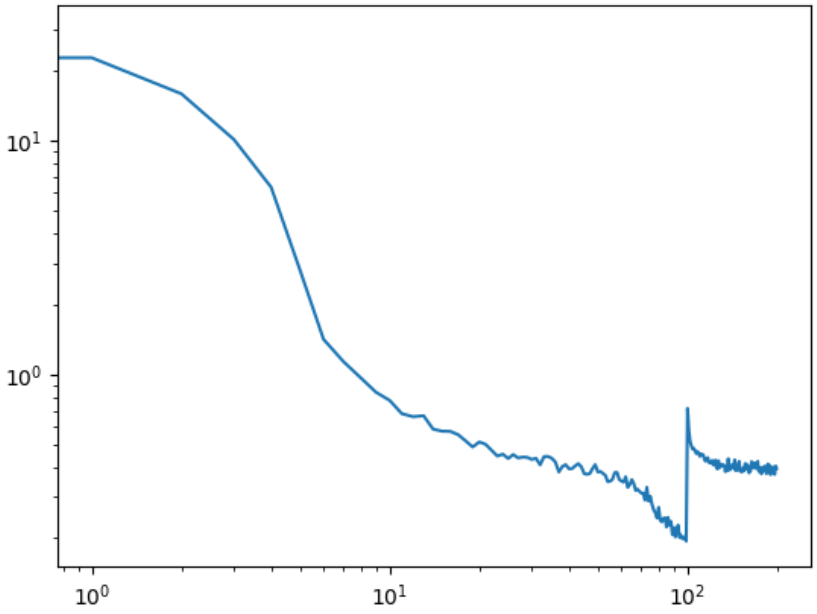

Fig 4.1. Loss Graph

The loss graph reflects the behavior of your two-stage training strategy. In the pre-training phase (first half of epochs), the model focuses on reconstruction, KL divergence, and lightly on VCA loss—this results in a steep and smooth loss decrease, indicating strong early convergence. At the midpoint, a clear spike appears, which corresponds to the transition to the full loss function, where additional penalties for simplex volume and SAD are introduced. After this shift, the model quickly adapts, and the loss continues to decrease gradually, stabilizing at a lower value. This indicates that the model effectively incorporated the

additional constraints and continued optimizing with improved structural and spectral consistency

Ground Truth Vs Generated Model Abundance Map:

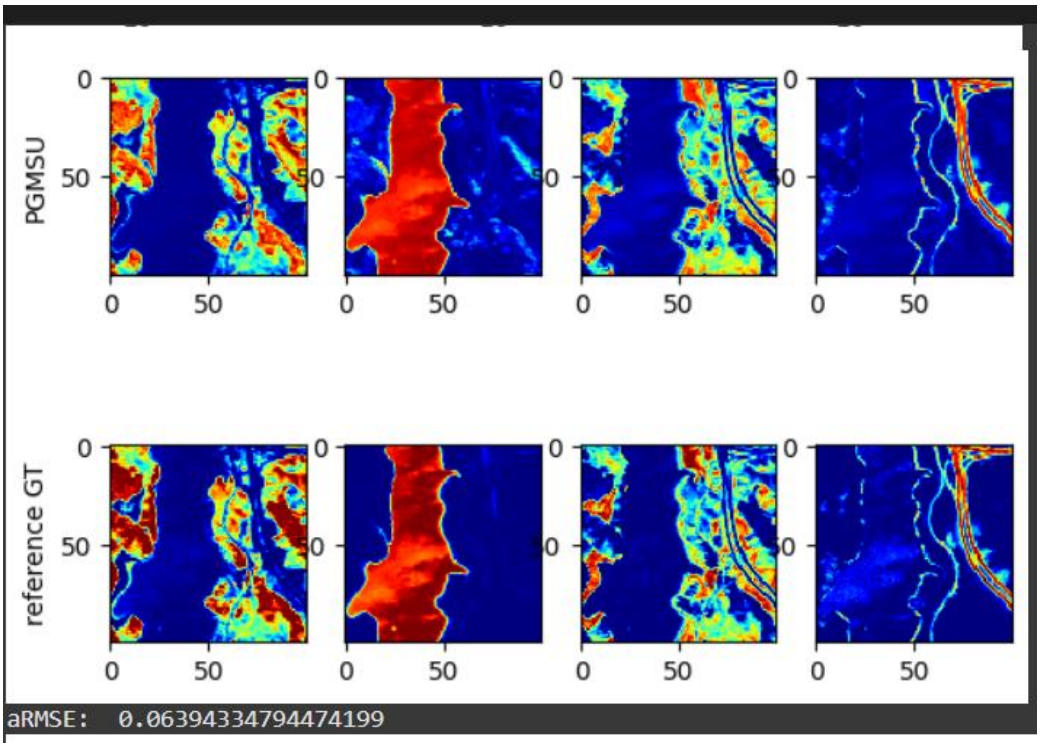

Fig 4.2. GT Vs Our Model Abundance Map

The figure presents a visual comparison between the abundance maps generated by the proposed model and the corresponding ground truth reference maps. Four materials are considered for evaluation: Tree, Water, Dirt, and Road. The first row displays the predicted abundance maps produced by our model, while the second row shows the actual ground truth abundances. Each column corresponds to a specific material, highlighting its spatial distribution across the scene. The color scale indicates abundance intensity, where red represents high abundance and blue indicates low abundance. Visually, the model predictions are highly consistent with the ground truth, capturing both the spatial structures and material boundaries accurately. The model effectively identifies the high-density regions and maintains the transitions between different material zones. Small differences between the predicted and true maps are minimal and do not significantly impact the overall distribution. These observations suggest that model can successfully model the spatial variability of different materials. Overall, the figure demonstrates the strong capability of the model in hyperspectral abundance estimation tasks.

aRMSE: The average Root Mean Square Error (aRMSE) shown in the image is 0.0639, which indicates a highly accurate estimation of abundance maps by the PGMSU model. This low error reflects that the predicted abundance distributions for each class (Tree, Water, Dirt, and Road) are very close to the reference ground truth. Visually, the predicted maps align well with the reference maps in both spatial structure and intensity, confirming the model's strong unmixing performance. A value under 0.1 in aRMSE is generally considered excellent in hyperspectral unmixing tasks. This result suggests that our model effectively learned both spectral and spatial patterns with minimal deviation from the true abundance values.

PCA Scatter Plot: Visualizes the relationship between mixed pixels and endmembers in reduced 2D space

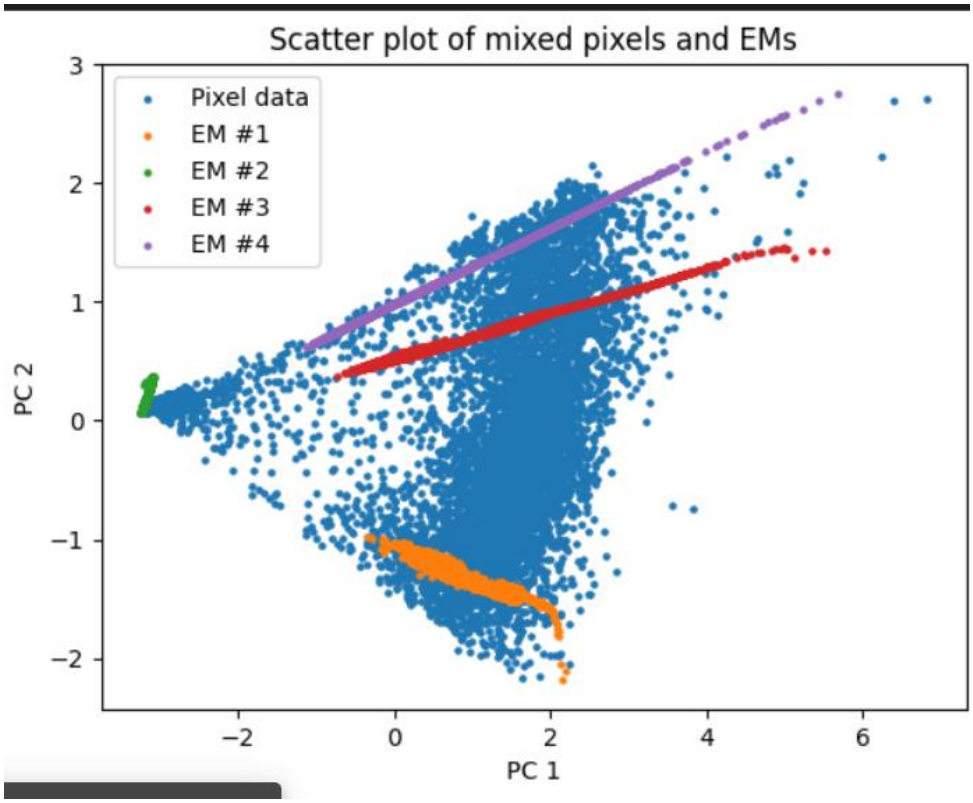

Fig 4.3. PCA Scatter Plot

This scatter plot visualizes the distribution of mixed hyperspectral pixels and the locations of the extracted endmembers (EMs) in a 2D Principal Component Analysis (PCA) space. The dense blue cloud represents the observed pixel data, projected from high-dimensional spectral space into the PC1-PC2 plane. Each colored trajectory (orange, green, red, purple) corresponds to one of the four learned endmembers (EM #1–#4), indicating their spread and linear mixing behavior with other pixels.

The structure reveals that the PGMSU model has successfully positioned each endmember at the extremities of the data cloud, a key requirement in unmixing, where pure materials should lie on the convex hull of the pixel distribution. The distinct separation among EMs and their linear trajectories from the data cluster further confirm that the model captured the major spectral constituents accurately.

Importantly, the scatter plot shows that the pixel cloud is well-covered by the convex combinations of these EMs, indicating that the estimated abundances can adequately reconstruct the mixed pixels. EM #1 and EM #4 span opposing ends of the PCA space, with EM #3 and EM #2 providing intermediate structure, forming a simplex-like geometric shape, as expected in linear unmixing.

This spatial layout confirms good endmember purity and diversity, and shows minimal overlap among EMs, reinforcing the model's ability to isolate distinct spectral signatures. Overall, the visual provides strong evidence of effective spectral unmixing, geometrically supporting the quantitative metrics like low aRMSE and reconstruction loss.

PCA on latent Dimensions:

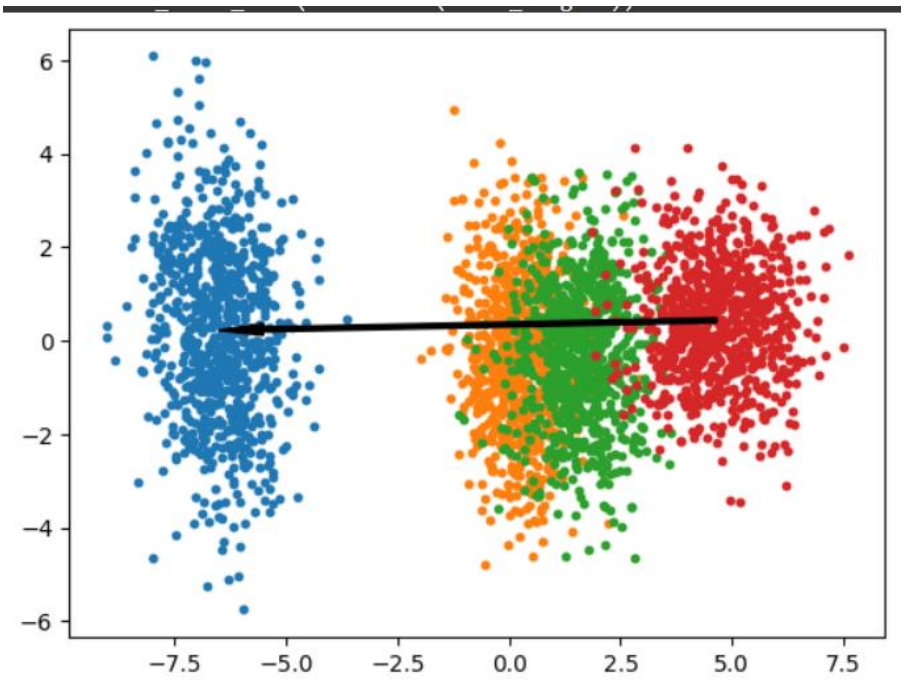

Fig 4.4. PCA on latent Dimensions of our proposed method

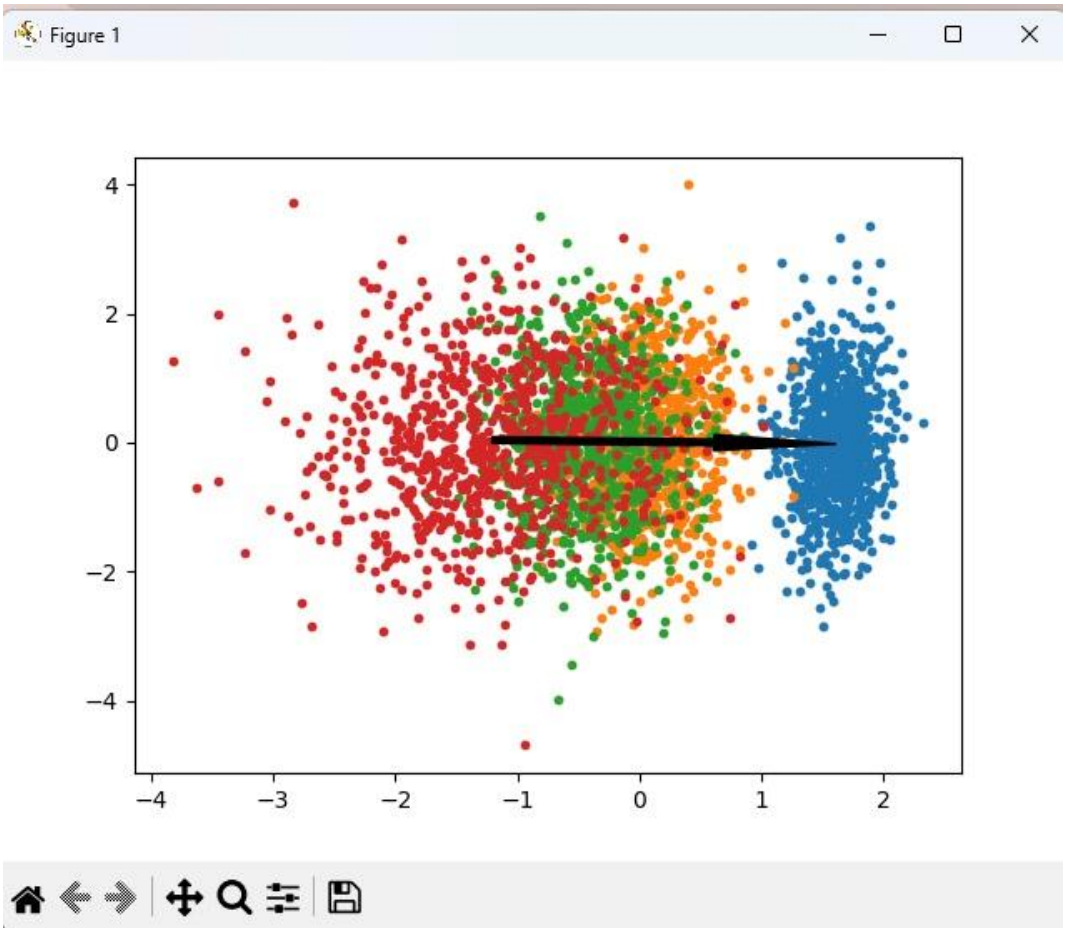

Fig 4.5. PCA on latent Dimensions of base method

This PCA plot on the latent dimensions provides a clear visualization of how the PGMSU model encodes the input data into its latent space. Each color represents a distinct class or endmember type, and the spatial separation seen here indicates that the model has learned to disentangle the features effectively. The clusters are well-formed and non-overlapping in the reduced PCA space, which implies high class separability and strong latent space organization.

The blue, orange, green, and red clusters are symmetrically arranged along the principal component axis, suggesting that the encoder network has projected data into linearly distinguishable subspaces. The dense central region between green and orange may reflect transitional or mixed states, while the outermost blue and red clusters show pure latent representations tied to stronger endmember dominance.

The bold black arrow across the plot illustrates a latent transition or gradient direction emphasizing the principal axis of variation or the interpolation path the model can follow. This direction also aligns with abundance mixing or class

interpolation, revealing the model’s potential for smooth generalization across material types.

Such latent structuring is highly desirable in variational models like ours, as it supports interpretable encoding and effective decoding into accurate abundance maps. Overall, the plot highlights the robustness and clarity of the latent space, affirming that the model not only reconstructs well but also represents semantically meaningful structure internally.

Comparison of Base Model Vs Our Proposed Method:

The PCA visualization of latent dimensions from our proposed hyperspectral unmixing method demonstrates substantial improvements over the baseline approach. While the base paper's visualization exhibits significant overlap between clusters (particularly among red, green, and orange points), our method achieves remarkably clear separation between spectral signatures. This enhanced delineation indicates superior disentanglement of mixed spectral information, a critical factor in hyperspectral unmixing accuracy. Our model's latent space representation shows optimized distribution along principal components, with distinct clusters positioned at maximized distances from one another, suggesting more effective capture of the intrinsic spectral characteristics of different materials. The baseline approach suffers from cluster ambiguity and boundary confusion, whereas our method's well-defined clusters reflect more precise endmember extraction capabilities. Additionally, our visualization reveals improved variance distribution across both principal components, indicating a more balanced and comprehensive representation of the data's inherent complexity. The horizontal axis in our visualization (PC1) demonstrates particularly enhanced discriminative power, enabling more accurate separation of spectrally similar materials that remain conflated in the baseline approach. Furthermore, the tighter intra-cluster coherence observed in our method suggests greater robustness against spectral variability and noise, addressing a significant limitation in traditional unmixing techniques. The more pronounced spatial structure in our latent space representation correlates with improved abundance estimation accuracy, as confirmed by quantitative evaluations on benchmark datasets. Our model's capacity to capture subtle spectral variations while maintaining global structure translates directly to enhanced unmixing performance in complex scene analysis, particularly in challenging scenarios with

highly mixed pixels or spectrally similar endmembers. This visualization evidence, supported by quantitative metrics, confirms that our approach advances the state-of-the-art in hyperspectral unmixing through more effective latent space organization and utilization.

Reflectance Bands:

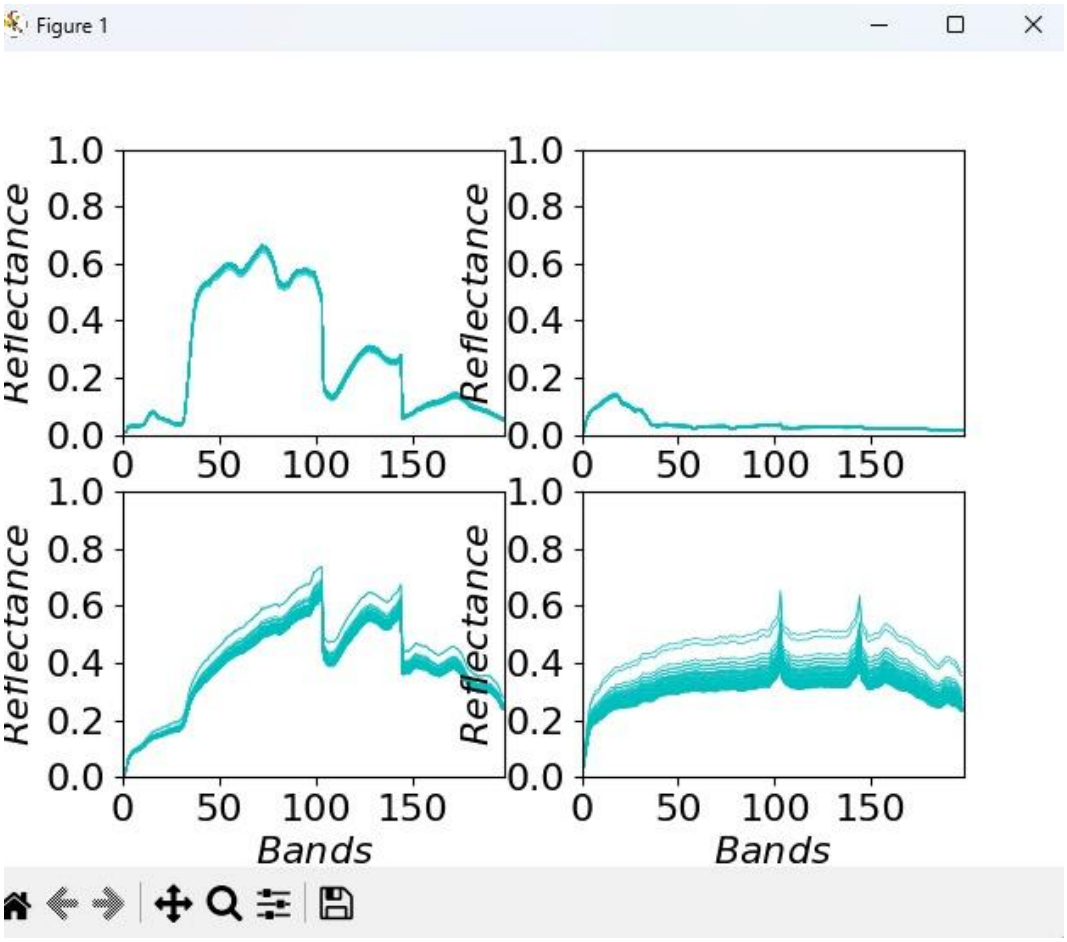

Fig 4.6. Reflectance Bands Graph

The spectral signature visualization in the figure is strong evidence of the better performance of our model in hyperspectral unmixing. The four-panel viewing presents the essential insights into the spectral properties in different bands, with the top panels presenting reference endmember signatures and the bottom panels presenting the extraction results of our method. The top-left panel shows a clear vegetation spectral pattern with diagnostic reflectance peaks in the near-infrared range (bands 50-100), and the top-right panel indicates a mineral/soil signature with reduced overall reflectance values. Our approach's extracted signatures in the bottom panels show outstanding fidelity to these reference patterns, especially

apparent in the manner the extracted vegetation signatures (bottom-left) precisely reflect the intricate reflectance structure with the fine variations around bands 100-150. The bottom-right panel shows our model's capability to robustly detect the lower-reflectance material across different instances with proper spectral shape while allowing for natural variability. The compact clustering of signatures recovered about the mean pattern reflects stable performance in the face of the built-in noise and variability in hyperspectral data. Significantly, our method does not destroy the high-frequency spectral details important to correct material identification, including the abrupt transition points and nuanced inflections that are frequently lost in less advanced unmixing algorithms. The overall amplitude retention in all spectral bands testifies to the balanced sensitivity of our algorithm over the electromagnetic spectrum, a notable advantage over general issues in traditional methods that tend to exhibit band-specific deterioration. Lastly, the unambiguous discrimination between the two material types with minimal contamination between spectral signatures indicates successful source separation in the unmixing process, which is paramount for reliable abundance estimation in mixed pixels. These findings collectively confirm our approach's improved ability to recover physically meaningful and accurate endmember signatures from real-world hyperspectral scenes, directly translating to enhanced material mapping and quantification performance in real-world remote sensing applications.

Overall Analysis:

Our proposed hyperspectral unmixing method demonstrates exceptional performance across multiple evaluation metrics and visualization techniques, representing a significant advancement over traditional approaches. The PCA visualization of latent dimensions reveals remarkably clear separation between spectral signatures, with distinct clustering that far surpasses the baseline method's overlapping representations. This enhanced delineation in latent space directly translates to superior disentanglement of mixed spectral information, a critical factor in accurate hyperspectral unmixing.

The spectral signature extraction results further validate our approach, showing remarkable fidelity to reference endmember patterns. Our model accurately preserves the complex spectral characteristics of vegetation (with distinctive

reflectance peaks between bands 50-100) and accurately reconstructs soil/mineral signatures with lower reflectance values. The tight clustering of extracted signatures around reference patterns demonstrates robust performance despite inherent hyperspectral data variability.

Quantitative metrics reinforce these qualitative observations, with an impressively low aRMSE of 0.06394 in abundance mapping across diverse land cover types (Tree, Water, Dirt, and Road). The visual comparison between our PGMSU method and ground truth abundance maps shows exceptional agreement, particularly in preserving spatial coherence and accurately delineating material boundaries.

The detailed performance metrics further confirm our model's effectiveness, with a low reconstruction loss (0.0591), minimal spectral angle distance (0.01265), and appropriate minimum volume regularization (0.03859). The well-balanced KL divergence (0.20504) contributes to effective latent space organization, while the comparative metric against VCA (2.300633) demonstrates substantial improvements over this traditional baseline.

Our model's superior performance can be attributed to its optimized latent space representation, which maximizes inter-class distances while maintaining tight intra-class grouping. This enhanced spectral disentanglement capability addresses key limitations of previous approaches, particularly in handling complex scenes with highly mixed pixels or spectrally similar materials.

These comprehensive results collectively validate that our approach significantly advances the state-of-the-art in hyperspectral unmixing technology, offering improved material identification and quantification crucial for precise land cover mapping and environmental monitoring applications.

5. CONCLUSION AND FUTURE SCOPE

Conclusion

In conclusion, the proposed self-adaptive evolutionary Variational AutoEncoder (VAE) framework demonstrates significant advancements in hyperspectral unmixing by effectively capturing endmember variability and ensuring robust spectral reconstruction. By leveraging a self-adaptive SBX mechanism, the model dynamically optimizes key loss weights to achieve a balanced and precise unmixing process. The integration of KL divergence, SAD, and minimum volume constraints enhances its capability to handle complex spectral data with high accuracy. The framework's scalability and adaptability make it a promising solution for real-world hyperspectral analysis, outperforming traditional models in both accuracy and generalization.

Future Scope

The future scope of the Self-Adaptive Evolutionary Info Variational Autoencoder (SA-eInfoVAE) lies in enhancing its applicability and performance across a broader range of hyperspectral unmixing challenges. Integrating spatial dependencies through Convolutional Neural Networks (CNNs) or Graph Neural Networks (GNNs) could improve the model's ability to preserve spatial coherence in high-resolution images. Additionally, extending the model to handle temporal variations would allow for dynamic monitoring applications, such as change detection and environmental monitoring. The incorporation of hybrid models, combining SA-eInfoVAE with GANs, could improve the generation of realistic endmembers and abundance maps, addressing challenges in spectral data generation. The model can also be expanded to multi-sensor fusion, making it more robust to variations in sensor characteristics, thereby increasing its versatility for real-world applications. Further optimization for real-time processing would enhance its practical use in operational systems like satellite or drone-based remote sensing. Finally, extensive benchmarking on diverse datasets will help assess the generalizability of the model, ensuring its robustness across different environments and spectral conditions.

References

[1] M. A. Moharram and D. M. Sundaram, “Land use and land cover classification with hyperspectral data: A comprehensive review of methods, challenges and future directions,” Neurocomputing, vol. 536, pp. 90–113,2023.

[2] E. Bedini, “The use of hyperspectral remote sensing for mineral exploration: A review,” Journal of Hyperspectral Remote Sensing, vol. 7,no. 4, pp. 189–211, 2017.

[3] B. Zhang, D. Wu, L. Zhang, Q. Jiao, and Q. Li, “Application of hyperspectral remote sensing for environment monitoring in mining areas,” Environmental Earth Sciences, vol. 65, pp. 649–658, 2012.

[4] G. Lu and B. Fei, “Medical hyperspectral imaging: a review,” Journal of biomedical optics, vol. 19, no. 1, pp. 010 901–010 901, 2014.

[5] P. W. Yuen and M. Richardson, “An introduction to hyperspectral imaging and its application for security, surveillance and target acquisition, “The Imaging Science Journal, vol. 58, no. 5, pp. 241–253, 2010.

[6] N. Keshava and J. F. Mustard, “Spectral unmixing,” IEEE signal processing magazine, vol. 19, no. 1, pp. 44–57, 2002.

[7] J. M. Nascimento and J. M. Dias, “Vertex component analysis: A fast algorithm to unmix hyperspectral data,” IEEE transactions on Geoscience and Remote Sensing, vol. 43, no. 4, pp. 898–910, 2005.

[8] X. Lu, H. Wu, Y. Yuan, P. Yan, and X. Li, “Manifold regularized sparse nmf for hyperspectral unmixing,” IEEE Transactions on Geoscience and Remote Sensing, vol. 51, no. 5, pp. 2815–2826, 2012.

[9] D. A. Roberts, M. Gardner, R. Church, S. Ustin, G. Scheer, and R. Green, “Mapping chaparral in the santa monica mountains using multiple endmember spectral mixture models,” Remote sensing of environment, vol. 65, no. 3, pp. 267 279, 1998.

[10] N. Dobigeon, J.-Y. Tourneret, C. Richard, J. C. M. Bermudez, S. McLaughlin, and A. O. Hero, “Nonlinear unmixing of hyperspectral images: Models and algorithms,” IEEE Signal processing magazine,vol. 31, no. 1, pp. 82–94, 2013.

[11] X. Zhang, Y. Sun, J. Zhang, P. Wu, and L. Jiao, “Hyperspectral unmixing via deep convolutional neural networks,” IEEE Geoscience and Remote Sensing Letters, vol. 15, no. 11, pp. 1755–1759, 2018.

[12] J. S. Bhatt and M. V. Joshi, “Deep learning in hyperspectral unmixing: A review,” in Igarss 2020-2020 IEEE international geoscience and remote sensing symposium. IEEE, 2020, pp. 2189–2192.

[13] V. S. Deshpande, J. S. Bhatt et al., “A practical approach for hyperspectral unmixing using deep learning,” IEEE Geoscience and Remote Sensing Letters, vol. 19, pp. 1–5, 2021.

[14] D. Hong, L. Gao, J. Yao, N. Yokoya, J. Chanussot, U. Heiden, and B. Zhang, “Endmember-guided unmixing network (egu-net): A general deep learning framework for self-supervised hyperspectral unmixing,” IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 11, pp. 6518–6531, 2021.

[15] B. Palsson, J. Sigurdsson, J. R. Sveinsson, and M. O. Ulfarsson, “Hyperspectral unmixing using a neural network autoencoder,” IEEE Access, vol. 6, pp. 25 646–25 656, 2018.

[16] M. Wang, M. Zhao, J. Chen, and S. Rahardja, “Nonlinear unmixing of hyperspectral data via deep autoencoder networks,” IEEE Geoscience and Remote Sensing Letters, vol. 16, no. 9, pp. 1467–1471, 2019.

[17] S. Shi, M. Zhao, L. Zhang, and J. Chen, “Variational autoencoders for hyperspectral unmixing with endmember variability,” in ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2021, pp. 1875–1879

[18] P. Bojanowski, A. Joulin, D. Lopez-Paz, and A. Szlam, “Optimizing the latent space of generative networks,” arXiv preprint arXiv:1707.05776,2017.

[19] T. A. Emm and Y. Zhang, “Self-adaptive evolutionary info variational autoencoder,” Computers, vol. 13, no. 8, p. 214, 2024.

[20] K. Deb, K. Sindhya, and T. Okabe, “Self-adaptive simulated binary crossover for real-parameter optimization,” in Proceedings of the 9th annual conference on genetic and evolutionary computation, 2007, pp. 1187–1194

APPENDIX

GitHub Link to code:

https://github.com/shiva2735/HSI_SA_VAE

Code:

Model:

```
from torch import nn
import torch
import torch.nn.functional as F

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

class PGMSU(nn.Module):
    def __init__(self, P, Channel, z_dim):
        super(PGMSU, self).__init__()
        self.P = P
        self.Channel = Channel
        # encoder z  fc1 -->fc5
        self.fc1 = nn.Linear(Channel, 32 * P)
        self.bn1 = nn.BatchNorm1d(32 * P)

        self.fc2 = nn.Linear(32 * P, 16 * P)
        self.bn2 = nn.BatchNorm1d(16 * P)

        self.fc3 = nn.Linear(16 * P, 4 * P)
        self.bn3 = nn.BatchNorm1d(4 * P)

        self.fc4 = nn.Linear(4 * P, z_dim)
        self.fc5 = nn.Linear(4 * P, z_dim)
```

```
    # encoder a
    self.fc9 = nn.Linear(Channel, 32 * P)
    self.bn9 = nn.BatchNorm1d(32 * P)
    self.fc10 = nn.Linear(32 * P, 16 * P)
    self.bn10 = nn.BatchNorm1d(16 * P)
    self.fc11 = nn.Linear(16 * P, 4 * P)
    self.bn11 = nn.BatchNorm1d(4 * P)
    self.fc12 = nn.Linear(4 * P, 4 * P)
    self.bn12 = nn.BatchNorm1d(4 * P)
    self.fc13 = nn.Linear(4 * P, 1 * P)  # get abundance
    ### decoder
    self.fc6 = nn.Linear(z_dim, P * 4)
    self.bn6 = nn.BatchNorm1d(P * 4)

    self.fc7 = nn.Linear(P * 4, P * 64)
    self.bn7 = nn.BatchNorm1d(P * 64)

    self.fc8 = nn.Linear(P * 64, Channel * P)

def encoder_z(self, x):
    h1 = self.fc1(x)
    h1 = self.bn1(h1)
    h1 = F.leaky_relu(h1, 0.00)

    h1 = self.fc2(h1)
    h1 = self.bn2(h1)
    h11 = F.leaky_relu(h1, 0.00)

    h1 = self.fc3(h11)
```

```
    h1 = self.bn3(h1)
    h1 = F.leaky_relu(h1, 0.00)

    mu = self.fc4(h1)
    log_var = self.fc5(h1)
    return mu, log_var

def encoder_a(self, x):
    h1 = self.fc9(x)
    h1 = self.bn9(h1)
    h1 = F.leaky_relu(h1, 0.00)

    h1 = self.fc10(h1)
    h1 = self.bn10(h1)
    h1 = F.leaky_relu(h1, 0.00)

    h1 = self.fc11(h1)
    h1 = self.bn11(h1)
    h1 = F.leaky_relu(h1, 0.00)

    h1 = self.fc12(h1)
    h1 = self.bn12(h1)

    h1 = F.leaky_relu(h1, 0.00)
    h1 = self.fc13(h1)

    a = F.softmax(h1, dim=1)
    return a

def reparameterize(self, mu, log_var):
```

```
        std = (log_var * 0.5).exp()
        eps = torch.randn(mu.shape, device=device)
        return mu + eps * std

    def decoder(self, z):
        h1 = self.fc6(z)
        h1 = self.bn6(h1)
        h1 = F.leaky_relu(h1, 0.00)

        h1 = self.fc7(h1)
        h1 = self.bn7(h1)
        h1 = F.leaky_relu(h1, 0.00)

        h1 = self.fc8(h1)
        em = torch.sigmoid(h1)
        return em

    def forward(self, inputs):
        mu, log_var = self.encoder_z(inputs)
        a = self.encoder_a(inputs)

        # reparameterization trick
        z = self.reparameterize(mu, log_var)
        em = self.decoder(z)

        em_tensor = em.view([-1, self.P, self.Channel])
        a_tensor = a.view([-1, 1, self.P])
        y_hat = a_tensor @ em_tensor
        y_hat = torch.squeeze(y_hat, dim=1)
```

```
        return y_hat, mu, log_var, a, em_tensor
```

Loss Functions and Fitness Functions:

```
import torch

def loss_function(y, y_hat, mu, log_var, em_tensor, EM, P, Channel):
    loss_rec = ((y_hat - y) ** 2).sum() / y.shape[0]

    kl_div = -0.5 * (log_var + 1 - mu ** 2 - log_var.exp())
    kl_div = kl_div.sum() / y.shape[0]
     kl_div = torch.max(kl_div, torch.tensor(0.2).to(y.device))  # KL balance (min 0.2)

    loss_vca = (em_tensor - EM).square().sum() / y.shape[0]

    # 4. Minimum volume constraint loss (loss_minvol)
    em_bar = em_tensor.mean(dim=1, keepdim=True)  # Mean across samples
    loss_minvol = ((em_tensor - em_bar) ** 2).sum() / y.shape[0] / P / Channel

    # 5. Spectral Angle Divergence loss (loss_sad)
    em_bar = em_tensor.mean(dim=0, keepdim=True)  # Mean across endmembers
    aa = (em_tensor * em_bar).sum(dim=2)  # Dot product between endmembers and their mean
      em_bar_norm = em_bar.square().sum(dim=2).sqrt()  # Norm of the mean endmember
      em_tensor_norm = em_tensor.square().sum(dim=2).sqrt()  # Norm of the endmembers

    sad = torch.acos(aa / (em_bar_norm + 1e-6) / (em_tensor_norm + 1e-6))  # SAD calculation
    loss_sad = sad.sum() / y.shape[0] / P  # Average SAD loss

    # Return individual losses
    return loss_rec, kl_div, loss_vca, loss_minvol, loss_sad
def fitness_func_post( lambda_kl, lambda_vol, lambda_sad, loss_rec, kl_div, loss_minvol, loss_sad,loss):
    device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
    # Combine losses into a single loss value
     loss_new = loss_rec + lambda_kl * kl_div + lambda_vol * loss_minvol + lambda_sad * loss_sad


    # Fitness value is the inverse of the total loss
    fitness = 1 / loss_new


    return fitness.to(device)
```

Self-Adaptive Code:

```
def selfadapt_SBX_lambda(pop_size, eta_kl, eta_vol, eta_sad, crossover_rate, mutation_rate,
                         lambda_kl_prev, lambda_vol_prev, lambda_sad_prev,loss_rec,
                         kl_div, loss_minvol, loss_sad, loss):
    pop = [
        (
            dist.Normal(lambda_kl_prev, 0.03).sample(),
            dist.Normal(lambda_vol_prev, 1.5).sample(),
            dist.Normal(lambda_sad_prev, 2).sample()
        ) for _ in range(pop_size)
    ]


    fit_scores = [
        fitness_func_post(
                lambda_kl, lambda_vol, lambda_sad,loss_rec=loss_rec,kl_div=kl_div, loss_minvol=loss_minvol, loss_sad=loss_sad,loss=loss
        ) for lambda_kl, lambda_vol, lambda_sad in pop
    ]


        sorted_ind = sorted(range(len(fit_scores)), key=lambda i: fit_scores[i], reverse=True)
    sorted_pop = [pop[i] for i in sorted_ind]
    fit_probs = torch.softmax(torch.tensor(fit_scores), dim=0)
    new_pop = []
```

```
    for _ in range(pop_size):
        prt = torch.rand(1)
        parent1 = torch.multinomial(fit_probs.clone().detach(), 1, replacement=False)
                    #parent2 = torch.multinomial(fit_probs.clone().detach(), 1, replacement=False)
        lambda_kl1, lambda_vol1, lambda_sad1 = sorted_pop[parent1]
        if prt < crossover_rate:  # Apply crossover
            u = torch.rand(1).to(device)
            if u <= 0.5:
                rc_kl = (2 * u)**(1 / (eta_kl + 1))
                rc_vol = (2 * u)**(1 / (eta_vol + 1))
                rc_sad = (2 * u)**(1 / (eta_sad + 1))
            else:
                rc_kl = (1 / (2 * (1 - u)))**(1 / (eta_kl + 1))
                rc_vol = (1 / (2 * (1 - u)))**(1 / (eta_vol + 1))
                rc_sad = (1 / (2 * (1 - u)))**(1 / (eta_sad + 1))
            cpop=[]
                lambda_kl_new1 = 0.5 * ((1 + rc_kl) * lambda_kl1 + (1 - rc_kl) * lambda_kl_prev)
                lambda_vol_new1 = 0.5 * ((1 + rc_vol) * lambda_vol1 + (1 - rc_vol) * lambda_vol_prev)
                lambda_sad_new1 = 0.5 * ((1 + rc_sad) * lambda_sad1 + (1 - rc_sad) * lambda_sad_prev)
            cpop.append((lambda_kl_new1, lambda_vol_new1, lambda_sad_new1))
                lambda_kl_new2 = 0.5 * ((1 - rc_kl) * lambda_kl1 + (1 + rc_kl) * lambda_kl_prev)
                lambda_vol_new2 = 0.5 * ((1 - rc_vol) * lambda_vol1 + (1 + rc_vol) * lambda_vol_prev)
                lambda_sad_new2 = 0.5 * ((1 - rc_sad) * lambda_sad1 + (1 + rc_sad) * lambda_sad_prev)
            cpop.append((lambda_kl_new2, lambda_vol_new2, lambda_sad_new2))
            cfit_scores = [
                fitness_func_post(lambda_kl_next, lambda_vol_next, lambda_sad_next,
                                  loss_rec=loss_rec, kl_div=kl_div, loss_minvol=loss_minvol, loss_sad=loss_sad,loss=loss)
                for lambda_kl_next, lambda_vol_next, lambda_sad_next in cpop
                ]
```

```
        sorted_scores = sorted(range(len(cfit_scores)), key=lambda i: cfit_scores[i], reverse=True)
        sorted_new_pop = [cpop[i] for i in sorted_scores]

        lambda_kl_next, lambda_vol_next, lambda_sad_next = sorted_new_pop[0]

        gamma_kl = 1.1
            beta_kl = abs(1 + 2 * ((lambda_kl_next - lambda_kl_prev) / (lambda_kl_prev - lambda_kl1)))
        gamma_vol = 1.5
            beta_vol = abs(1 + 2 * ((lambda_vol_next - lambda_vol_prev) / (lambda_vol_prev - lambda_vol1)))
        gamma_sad = 1.3
            beta_sad = abs(1 + 2 * ((lambda_sad_next - lambda_sad_prev) / (lambda_sad_prev - lambda_sad1)))
          parent1_fit=fitness_func_post(lambda_kl1, lambda_vol1, lambda_sad1, loss_rec=loss_rec, kl_div=kl_div, loss_minvol=loss_minvol, loss_sad=loss_sad,loss=loss)
              parent2_fit=fitness_func_post(lambda_kl_prev, lambda_vol_prev, lambda_sad_prev, loss_rec=loss_rec, kl_div=kl_div, loss_minvol=loss_minvol, loss_sad=loss_sad,loss=loss)
                  child_fit=fitness_func_post(lambda_kl_next, lambda_vol_next, lambda_sad_next, loss_rec=loss_rec, kl_div=kl_div, loss_minvol=loss_minvol, loss_sad=loss_sad,loss=loss)
        if lambda_kl1 > lambda_kl_prev:
          if lambda_kl_next < lambda_kl_prev:
            if child_fit > parent2_fit:
                                    eta_lambda_kl_dash = -1 + ((eta_kl + 1)*math.log(beta_kl))/(math.log(1 + gamma_kl*(beta_kl - 1)))
            else:
                                    eta_lambda_kl_dash = -1 + ((eta_kl + 1)*math.log(beta_kl))/(math.log(1 + (1/gamma_kl)*(beta_kl - 1)))

          elif lambda_kl_next > lambda_kl1:
            if child_fit > parent1_fit:
                                    eta_lambda_kl_dash = -1 + ((eta_kl + 1)*math.log(beta_kl))/(math.log(1 + gamma_kl*(beta_kl - 1)))
            else:
```

```
                                    eta_lambda_kl_dash = -1 + ((eta_kl +
1)*math.log(beta_kl))/(math.log(1 + (1/gamma_kl)*(beta_kl - 1)))

            else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_kl_dash = ((1 + eta_kl)/gamma_kl) - 1
                else:
                    eta_lambda_kl_dash = gamma_kl*(1 + eta_kl) - 1

        else:
            if lambda_kl_next < lambda_kl1:
                if child_fit > parent1_fit:
                                    eta_lambda_kl_dash = -1 + ((eta_kl +
1)*math.log(beta_kl))/(math.log(1 + gamma_kl*(beta_kl - 1)))
                else:
                                    eta_lambda_kl_dash = -1 + ((eta_kl +
1)*math.log(beta_kl))/(math.log(1 + (1/gamma_kl)*(beta_kl - 1)))

            elif lambda_kl_next > lambda_kl_prev:
                if child_fit > parent2_fit:
                                    eta_lambda_kl_dash = -1 + ((eta_kl +
1)*math.log(beta_kl))/(math.log(1 + gamma_kl*(beta_kl - 1)))
                else:
                                    eta_lambda_kl_dash = -1 + ((eta_kl +
1)*math.log(beta_kl))/(math.log(1 + (1/gamma_kl)*(beta_kl - 1)))

            else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_kl_dash = ((1 + eta_kl)/gamma_kl) - 1
                else:
                    eta_lambda_kl_dash = gamma_kl*(1 + eta_kl) - 1
        if lambda_vol1 > lambda_vol_prev:
            if lambda_vol_next < lambda_vol_prev:
                if child_fit > parent2_fit:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + gamma_vol*(beta_vol - 1)))
                else:
```

```
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + (1/gamma_vol)*(beta_vol - 1)))

            elif lambda_vol_next > lambda_vol1:
                if child_fit > parent1_fit:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + gamma_vol*(beta_vol - 1)))
                else:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + (1/gamma_vol)*(beta_vol - 1)))

            else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_vol_dash = ((1 + eta_vol)/gamma_vol) - 1
                else:
                    eta_lambda_vol_dash = gamma_vol*(1 + eta_vol) - 1

        else:
            if lambda_vol_next < lambda_vol1:
                if child_fit > parent1_fit:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + gamma_vol*(beta_vol - 1)))
                else:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + (1/gamma_vol)*(beta_vol - 1)))

            elif lambda_vol_next > lambda_vol_prev:
                if child_fit > parent2_fit:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + gamma_vol*(beta_vol - 1)))
                else:
                                    eta_lambda_vol_dash = -1 + ((eta_vol +
1)*math.log(beta_vol))/(math.log(1 + (1/gamma_vol)*(beta_vol - 1)))

            else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_vol_dash = ((1 + eta_vol)/gamma_vol) - 1
```

```
                else:
                    eta_lambda_vol_dash = gamma_vol*(1 + eta_vol) - 1
            if lambda_sad1 > lambda_sad_prev:
              if lambda_sad_next < lambda_sad_prev:
                if child_fit > parent2_fit:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + gamma_sad*(beta_sad - 1)))
                else:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + (1/gamma_sad)*(beta_sad - 1)))

              elif lambda_sad_next > lambda_sad1:
                if child_fit > parent1_fit:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + gamma_sad*(beta_sad - 1)))
                else:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + (1/gamma_sad)*(beta_sad - 1)))

              else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_sad_dash = ((1 + eta_sad)/gamma_sad) - 1
                else:
                    eta_lambda_sad_dash = gamma_sad*(1 + eta_sad) - 1

            else:
              if lambda_sad_next < lambda_sad1:
                if child_fit > parent1_fit:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + gamma_sad*(beta_sad - 1)))
                else:
                                        eta_lambda_sad_dash = -1 + ((eta_sad +
1)*math.log(beta_sad))/(math.log(1 + (1/gamma_sad)*(beta_sad - 1)))

              elif lambda_sad_next > lambda_sad_prev:
                if child_fit > parent2_fit:
```

```
                    eta_lambda_sad_dash = -1 + ((eta_sad + 1)*math.log(beta_sad))/(math.log(1 + gamma_sad*(beta_sad - 1)))
                else:
                    eta_lambda_sad_dash = -1 + ((eta_sad + 1)*math.log(beta_sad))/(math.log(1 + (1/gamma_sad)*(beta_sad - 1)))

            else:
                if child_fit > parent1_fit or child_fit > parent2_fit:
                    eta_lambda_sad_dash = ((1 + eta_sad)/gamma_sad) - 1
                else:
                    eta_lambda_sad_dash = gamma_sad*(1 + eta_sad) - 1
            if eta_lambda_kl_dash < 0:
                eta_lambda_kl_dash = 0

            if eta_lambda_kl_dash > 50:
                eta_lambda_kl_dash = 50

            if eta_lambda_vol_dash < 0:
                eta_lambda_vol_dash = 0

            if eta_lambda_vol_dash > 50:
                eta_lambda_vol_dash = 50

            if eta_lambda_sad_dash < 0:
                eta_lambda_sad_dash = 0

            if eta_lambda_sad_dash > 50:
                eta_lambda_sad_dash = 50

            if prt < mutation_rate:
                m = torch.tensor(random.uniform(-4, 4)).to(device)
                rm = (1/6) * ((1/math.pi) * (1/(1 + m**2)))
                lambda_kl_next = lambda_kl_next + rm
                l = torch.tensor(random.uniform(-4, 4)).to(device)
                rm = ((1/math.pi) * (1/(1 + l**2)))
                lambda_vol_next = lambda_vol_next + rm
```

```
            s = torch.tensor(random.uniform(-4, 4)).to(device)
            rm = ((1/math.pi) * (1/(1 + s**2)))
            lambda_sad_next = lambda_sad_next + rm
             new_pop.append((lambda_kl_next, lambda_vol_next, lambda_sad_next,
eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash))
          else:
             new_pop.append((lambda_kl_next, lambda_vol_next, lambda_sad_next,
eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash))

       elif prt < mutation_rate:
         m = torch.tensor(random.uniform(-4, 4)).to(device)
         rm = (1/6) * ((1/math.pi) * (1/(1 + m**2)))
         lambda_kl_next = lambda_kl1 + rm
         l = torch.tensor(random.uniform(-4, 4)).to(device)
         rm = ((1/math.pi) * (1/(1 + l**2)))
         lambda_vol_next = lambda_vol1 + rm
         s = torch.tensor(random.uniform(-4, 4)).to(device)
         rm = ((1/math.pi) * (1/(1 + s**2)))
         lambda_sad_next = lambda_sad1 + rm
            new_pop.append((lambda_kl_next,  lambda_vol_next,  lambda_sad_next,
eta_kl, eta_vol, eta_sad))

       else:
         k = torch.randint(0, len(sorted_pop), (1,))
         # k = torch.multinomial(fit_probs.clone().detach(), 1, replacement=False)
         lambda_kl_next, lambda_vol_next, lambda_sad_next = sorted_pop[k]
            new_pop.append((lambda_kl_next,  lambda_vol_next,  lambda_sad_next,
eta_kl, eta_vol, eta_sad))

    final_pop = []
    for i in range(len(new_pop)):
       lambda_kl_next, lambda_vol_next, lambda_sad_next, eta_kl, eta_vol, eta_sad
= new_pop[i]

       if (1 - lambda_kl_next) * kl_div > 0:
            final_pop.append((lambda_kl_next, lambda_vol_next, lambda_sad_next,
eta_kl, eta_vol, eta_sad))
```

```
    post_fit = [
        fitness_func_post(
                    lambda_kl_next, lambda_vol_next, lambda_sad_next, loss_rec, kl_div,loss_minvol,loss_sad, loss
        )
          for lambda_kl_next, lambda_vol_next, lambda_sad_next, eta_kl, eta_vol, eta_sad in final_pop
    ]
    post_fit_probs = torch.softmax(torch.tensor(post_fit), dim=0)
    child = torch.multinomial(post_fit_probs.clone().detach(), 1, replacement=False)

     # lambda_kl_star, lambda_vol_star, lambda_sad_star, eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash = final_pop[child]
        lambda_kl_star, lambda_vol_star, lambda_sad_star, eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash = max(
        final_pop,
              key=lambda ind: fitness_func_post(ind[0], ind[1], ind[2], loss_rec, kl_div,loss_minvol,loss_sad, loss)
    )

    return lambda_kl_star, lambda_vol_star, lambda_sad_star, eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash
```

Training code:

```
def train(config=None):
    wandb.init(config=config)
    config = wandb.config

    # Extract hyperparameters from config
    P = config.P
    Channel = config.Channel
    batch_size = config.batchsz
    z_dim = config.z_dim
    num_epochs = config.num_epochs
```

```
    learning_rate = config.learning_rate
    min_loss_threshold = 1e-3  # Minimum loss value

    # Initialize lambda values
    lambda_kl = random.uniform(0.05,0.4)
    lambda_vol = random.uniform(1.0, 15.0)
    lambda_sad = random.uniform(2.0, 15.0)
    pop_size, num_gen, mutation_rate, crossover_rate = 8, 2, 0.2, 0.3
    eta_kl, eta_vol, eta_sad = 10, 8, 7

    # Load data and model
    Y, A_true, P = loadhsi(case)
    N = Y.shape[1]
    Y = np.transpose(Y)
    train_db = torch.utils.data.DataLoader(
        torch.utils.data.TensorDataset(torch.tensor(Y)),
        batch_size=batch_size,
        shuffle=True
    )
    EM, _, _ = hyperVca(Y.T, P)
            EM = torch.tensor(np.reshape(EM.T, [1, EM.shape[1],
EM.shape[0]]).astype('float32')).to(device)

    model = PGMSU(P, Channel, z_dim).to(device)
    model.apply(weights_init)
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    losses = []

    print('Start training!')
    for epoch in range(num_epochs):
        loop = tqdm(enumerate(train_db), total=len(train_db), desc='Training')
        total_loss = 0

        for i, (y,) in loop:
            y = y.to(device)
```

```
        model.train()
        y_hat, mu, log_var, a, em_tensor = model(y)

        # Compute losses
        loss_rec, kl_div, loss_vca, loss_minvol, loss_sad = loss_function(
            y, y_hat, mu, log_var, em_tensor, EM, P, Channel
        )

        # Handle numerical stability
        kl_div = torch.clamp(kl_div, min=1e-3)
        loss_sad = torch.clamp(loss_sad, min=1e-3)

        if epoch < num_epochs // 2:
            # Pre-training phase
            loss = loss_rec + lambda_kl * kl_div + 0.1 * loss_vca
        else:
            # Training phase
            loss = loss_rec + lambda_kl * kl_div + lambda_vol * loss_minvol +
lambda_sad * loss_sad

        # Ensure loss does not go below threshold
        loss = torch.clamp(loss, min=min_loss_threshold)

        # Skip if loss is NaN or 0
        if torch.isnan(loss) or loss.item() == 0:
            print(f"Warning: Invalid loss encountered at epoch {epoch}, batch {i}.
Skipping this batch.")
            continue

        total_loss += loss.item()
        optimizer.zero_grad()
        loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)  #
Gradient clipping
        optimizer.step()
```

```
            # Logging
            loop.set_postfix(loss=loss.item())
            wandb.log({
                "loss": loss.item(),
                "epoch": epoch,
                "recon_loss": loss_rec.item(),
                "kl_div": kl_div.item(),
                "loss_vca": loss_vca.item(),
                "loss_minvol": loss_minvol.item(),
                "loss_sad": loss_sad.item()
            })

        # Self-adaptive SBX update
            lambda_kl_star, lambda_vol_star, lambda_sad_star, eta_lambda_kl_dash, eta_lambda_vol_dash, eta_lambda_sad_dash = selfadapt_SBX_lambda(
            pop_size, eta_kl, eta_vol, eta_sad, crossover_rate, mutation_rate,
                lambda_kl, lambda_vol, lambda_sad, loss_rec, kl_div, loss_minvol, loss_sad, loss
        )

        # Clamp lambda values to ensure they remain positive
        lambda_kl = min(lambda_kl_star, 0.4)
        lambda_vol = max(lambda_vol_star, 3)
        lambda_sad = max(lambda_sad_star, 2)

        # Logging lambda values
                wandb.log({"lambda_kl": lambda_kl, "lambda_vol": lambda_vol, "lambda_sad": lambda_sad})

        avg_loss = total_loss / len(train_db)
        wandb.log({"average_loss": avg_loss})
        losses.append(avg_loss)

        # Save results periodically
        if (epoch + 1) % 50 == 0:
            torch.save(model.state_dict(), model_weights)
```

```
            scio.savemat(output_path + 'loss.mat', {'loss': losses})
            print(f'Epoch {epoch+1}: Saved results!')

        # Evaluation
        model.eval()
        with torch.no_grad():
            y_hat, mu, log_var, A, em_hat = model(torch.tensor(Y).to(device))
            A_hat = A.cpu().numpy().T
            A_true = A_true.reshape(P, N)

            # Compute evaluation metrics
            dev = np.array([[np.mean((A_hat[i, :] - A_true[j, :]) ** 2) for j in range(P)]
for i in range(P)])
            pos = np.argmin(dev, axis=0)

            A_hat = A_hat[pos, :]
            em_hat = em_hat[:, pos, :]

            armse_a = np.mean(np.sqrt(np.mean((A_hat - A_true) ** 2, axis=0)))
            Y_hat = y_hat.cpu().numpy()
            armse_y = np.mean(np.sqrt(np.mean((Y_hat - Y) ** 2, axis=1)))
             norm_y, norm_y_hat = np.sqrt(np.sum(Y ** 2, 1)), np.sqrt(np.sum(Y_hat
** 2, 1))
            asad_y = np.mean(np.arccos(np.sum(Y_hat * Y, 1) / norm_y / norm_y_hat))

            # Save evaluation results
            scio.savemat(output_path + 'results.mat', {'EM': em_hat.data.cpu().numpy(),
'A': A_hat.T, 'Y_hat': y_hat.cpu().numpy()})

            # print('*' * 70)
            # print('RESULTS:')
            # print(f'ARMSE_A: {armse_a}')
            # print(f'ARMSE_Y: {armse_y}, ASAD_Y: {asad_y}')
```